第7章: Django・DRF バックエンド詳細 — サーバー側の作り（完全版）

> この章では、本アプリの「サーバー側（バックエンド）」を、**実際のソースコードを1行残らず読み解きながら** 学びます。フロントエンド（React）が画面を作る係なら、バックエンド（Django）は **データを守り、計算し、保存する「裏方の頭脳」** です。
>
> ブラウザの画面で「保存」ボタンを押したとき、その裏側で何が起きているのか。データはどこを通り、誰がチェックし、どうやってデータベースに書き込まれるのか。本章はその「リクエストの旅」を、本アプリの人員管理（member）機能を主役にして最後まで追いかけます。
>
> この章は長いですが、**辞書のように使えること** を目指しています。実務で「このAPIどう動いてるんだっけ」「履歴ってどこで記録されてるの」と思ったら、ここに戻って該当箇所を引いてください。

この章で学ぶこと

- **リクエストの旅** — ブラウザのボタン1つから、サーバーが応答を返すまでの全行程
- **プロジェクトとアプリの違い** — `hozen_another/`（プロジェクト）と `kosu/`（アプリ）の役割分担
- **MVT＋API構成** — モデル・ビュー・シリアライザ・URLの4役がどう連携するか
- `member_views.py` **全225行を1行ずつ完全解説** — 一覧(GET)・新規(POST)・更新(PUT)・削除(DELETE)
- `serializers.py` **全行解説** — Pythonオブジェクトとデータの相互変換
- `urls.py` **解説** — URLとビューを結びつける配線盤、`<int:pk>`・`as_view`・`static`
- `signals.py` **解説** — 保存・削除のたびに自動で「変更履歴」を残すしくみ（pre_save/post_save/post_delete/get_changes）
- `middleware/clear_session_middleware.py` — リクエストをスレッドに保管する CurrentRequest
- `apps.py` **の ready** — シグナルをアプリ起動時に有効化する1行
- `main_utils.py` **の CustomPagination** — 1ページ何件表示するかを動的に決めるページ送り
- `asynchronous_views.py` — 時間のかかるバックアップ/復元を「裏で」走らせる非同期処理
- `settings.py` — MIDDLEWARE・CORS・SESSION・REST_FRAMEWORK などプロジェクト全体の設定
- **HTTPメソッドとステータスコード・セッション認証・CSRF** の基礎

目次

0. 前提知識：バックエンドとは何者か
1. リクエストの旅 — ボタン1つから応答まで
2. プロジェクトとアプリ — hozen_another と kosu
3. モデル・ビュー・シリアライザ・URLの4役

4. HTTPメソッドとステータスコード
5. 実コード全解説：member_views.py 全225行
6. 実コード全解説：serializers.py
7. 実コード全解説：urls.py（配線盤）
8. 実コード全解説：CustomPagination（main_utils.py）
9. 実コード全解説：signals.py（自動の変更履歴）
10. 実コード全解説：middleware と apps.py
11. 実コード全解説：asynchronous_views.py（非同期処理）
12. 実コード全解説：settings.py（プロジェクト設定）
13. セッション認証とCSRF
14. トラブルシューティング
15. 演習問題
16. この章のまとめ

---

0. 前提知識：バックエンドとは何者か

「この章のコードが何をしているのか」を理解するために、まず **バックエンド** という言葉の意味から押さえます。

0.1 フロントとバックの役割分担

> **フロントエンド（front-end：フロントエンド）とは？** ユーザーの目の前（front）で動く部分。本アプリでは React で書かれた画面（ボタン・表・入力欄）。ブラウザの中で動きます。
>
> **バックエンド（back-end：バックエンド）とは？** 画面の裏（back）で動く部分。本アプリでは Django（ジャンゴ）というPythonの仕組みで書かれた「サーバー側のプログラム」。ユーザーの目には見えません。データベースに近いところで、データの保存・取得・計算・チェックを担当します。

レストランでたとえると：

登場人物	役割	本アプリでは
お客さん	注文する人	ブラウザを使う従業員
ウェイター（ホール）	注文を聞き、料理を運ぶ	フロントエンド（React）
厨房（キッチン）	実際に料理を作る	バックエンド（Django）
冷蔵庫・食材棚	材料の保管庫	データベース（PostgreSQL）

お客さん（ブラウザ）は厨房（Django）に直接入りません。必ずウェイター（React）を通して注文（リクエスト）を出し、料理（データ）を受け取ります。

なぜ分けるの？ 画面の見た目（フロント）と、データの管理（バック）を分けておくと、片方を直しても、もう片方を壊しにくいからです。たとえば「ボタンの色を変える」のはフロントだけの仕事。「保存できる人員の上限を変える」のはバックだけの仕事、というふうに役割が明確になります。

0.2 Django と DRF とは

Django（ジャンゴ）とは？ Python言語で「Webサーバー側のプログラム」を作るための道具一式（フレームワーク）。データベースの操作・URLの振り分け・セキュリティなど、Webアプリに必要な部品があらかじめ揃っています。「自分でゼロから作らなくていい家具付き物件」のようなものです。

DRF（ディーアールエフ）とは？ Django REST Framework（ジャンゴ・レスト・フレームワーク）の略。Django に「**API（エーピーアイ）**」を作る機能を足す追加部品。本アプリのバックエンドは、画面そのものを返すのではなく、**データ（JSON）だけを返すAPI** として作られています。その主役がこのDRFです。

API（Application Programming Interface：エーピーアイ）とは？ プログラム同士が会話するための「窓口」。本アプリでは「`/api/member_list/` にアクセスすると人員一覧データが返ってくる」といった、URLごとの窓口の集まりがAPIです。人間向けの画面ではなく、**プログラム（React）向けのデータ出口** だと考えてください。

JSON（ジェイソン）とは？ JavaScript Object Notation の略。データを文字列で表す共通フォーマット。`{"employee_no": 123, "name": "山田"}` のように「名前と値のペア」で書きます。フロントとバックは、このJSONという「共通語」でデータをやり取りします。第4章・第5章でも登場しました。

---

1. リクエストの旅 — ボタン1つから応答まで

本章の地図として、まず「リクエストが通る道」全体を俯瞰します。これを頭に入れておくと、後で個々のコードを読むとき「あ、これはあの段階の話だな」と位置づけられます。

リクエスト（request：リクエスト）とは？ ブラウザがサーバーに送る「お願い」。「人員一覧をください」「この人員を保存してください」などの依頼。

レスポンス（response：レスポンス）とは？ サーバーがリクエストに対して返す「返事」。データ本体（JSON）と、成否を表す番号（ステータスコード）が入っています。

1.1 旅の全行程（人員一覧を表示する例）

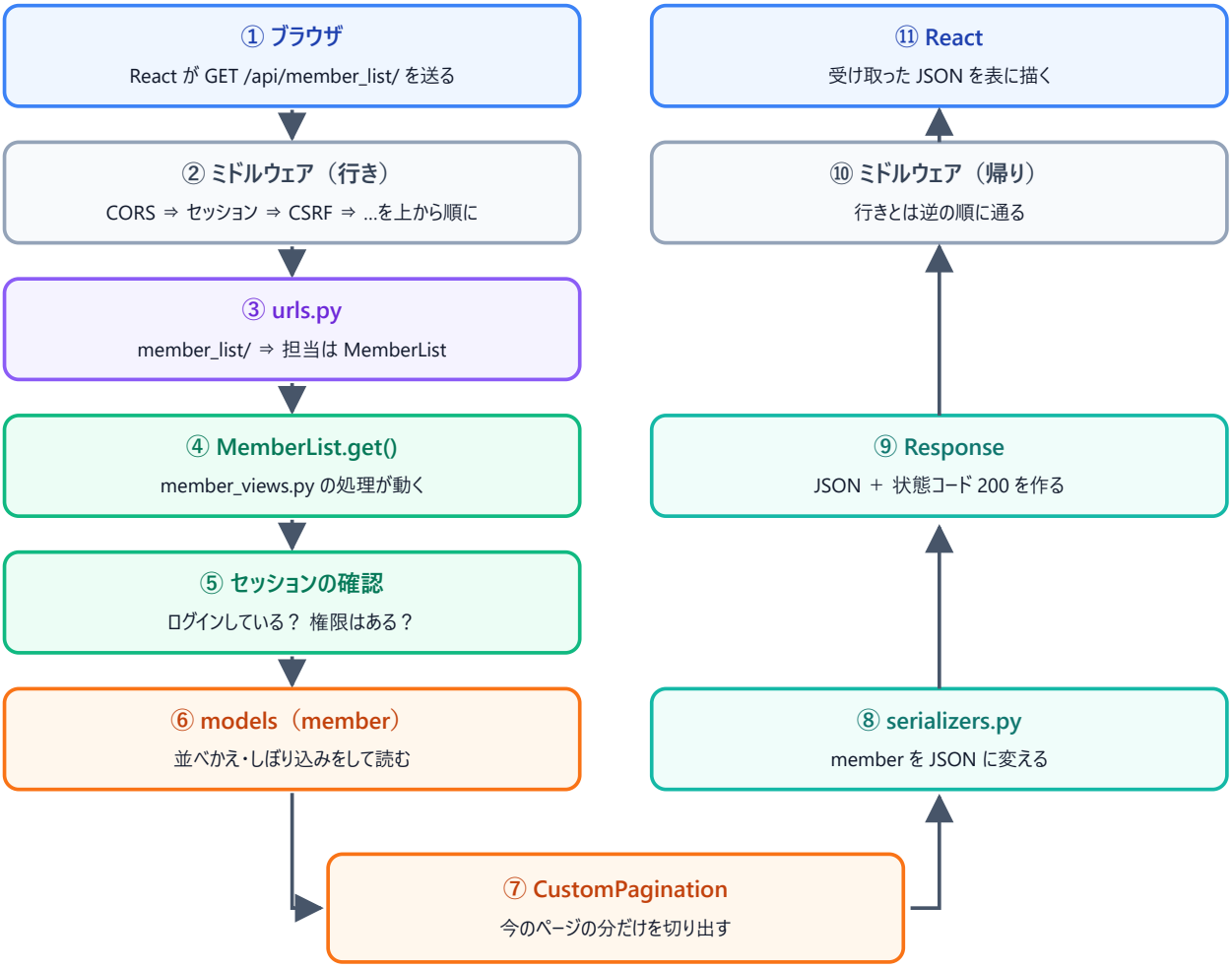


図7-1: 人員の一覧を開いた時の、リクエストの旅。左を下へ進み（①〜⑥）、ページ分けを通って（⑦）、右を上へ戻る（⑧〜⑪）

このうち、②③④⑤⑥⑦⑧⑨が「サーバー側（この章の範囲）」です。1つずつコードで確認していきます。

> **保存（POST）のときは？** 保存・更新・削除の場合は、⑥の前後で **signals.py（シグナル）** が割り込み、「誰が・どのデータを・どう変えたか」を自動で `History`（履歴）テーブルに記録します。これは§9で詳しく見ます。

---

2. プロジェクトとアプリ — hozen_another と kosu

Django には「プロジェクト」と「アプリ」という2つの単位があります。最初に必ず混乱するポイントなので、ここで整理します。

2.1 2つの違い

> **プロジェクト（project）とは？** アプリ全体をまとめる「箱」。設定ファイルや、サイト全体の入口を持ちます。本アプリでは `hozen_another/` フォルダがそれ。`settings.py`（全体設定）や `wsgi.py`（サーバー起動口）が入っています。
>
> **アプリ（app）とは？** 1つの機能のまとまり。本アプリでは `kosu/` フォルダがそれ。モデル・ビュー・URL・シグナルなど、工数管理という機能の中身が全部ここにあります。

たとえると：

Django用語	たとえ	本アプリ
プロジェクト	会社全体・本社ビル	`hozen_another/`
アプリ	部署（事業部）	`kosu/`

本アプリはアプリが `kosu` 1つだけのシンプルな構成です。大きなサービスだと、1つのプロジェクトに「ブログアプリ」「ショップアプリ」のように複数のアプリが入ることもあります。

2.2 kosu フォルダの中身（この章で扱うファイル）

コピー

```
kosu/                                ← アプリ本体
├── models.py                        ← データの設計図（第8章で詳説）
├── urls.py                          ← URLとビューの配線盤（§7）
├── apps.py                          ← アプリの起動設定（§10）
├── signals.py                       ← 保存/削除時の自動履歴（§9）
├── tasks.py                         ← 時間のかかる処理の中身（バックアップ等）
├── views/                           ← ビュー（処理の本体）の集まり
│   ├── member_views.py              ← 人員管理API（§5・この章の主役）
│   ├── serializers.py               ← データ⇔JSON変換係（§6）
│   ├── asynchronous_views.py        ← 非同期処理API（§11）
│   └── （kosu_views, team_views ...）  ← 他機能のビュー
├── utils/
│   └── main_utils.py                ← 便利関数・CustomPagination（§8）
└── middleware/
    └── clear_session_middleware.py  ← リクエストをスレッド保管（§10）
```

> **なぜ views がフォルダになっているの？** 普通の小さなアプリでは `views.py` という1ファイルですが、本アプリは機能が多いので、`views/` フォルダの中に機能ごとのファイル（member, kosu, team...）に分けています。中身は同じ「ビュー」です。

3. モデル・ビュー・シリアライザ・URLの4役

本アプリのバックエンドは、ざっくり4つの役者が連携して動いています。最初にこの「役割分担」を頭に入れておくと、コードがぐっと読みやすくなります。

3.1 4役の早見表

役者	ファイル	役割	レストランでたとえると
URL	`urls.py`	どのURLが来たら誰に任せるか決める	受付・案内係
ビュー（View）	`member_views.py` 等	実際の処理を行う頭脳	料理長（指揮を執る）
モデル（Model）	`models.py`	データベースとのやり取り	食材棚・在庫管理
シリアライザ（Serializer）	`serializers.py`	データ⇔JSON の翻訳	盛り付け・配膳の係

> **MVT（エムブイティー）とは？** Model（モデル）・View（ビュー）・Template（テンプレート）の頭文字。Djangoの基本構成。本アプリはテンプレート（HTML生成）の代わりにAPIを返すので、Templateの位置に **Serializer（シリアライザ）** が入る、と考えると分かりやすいです。

3.2 連携の流れ（保存を例に）

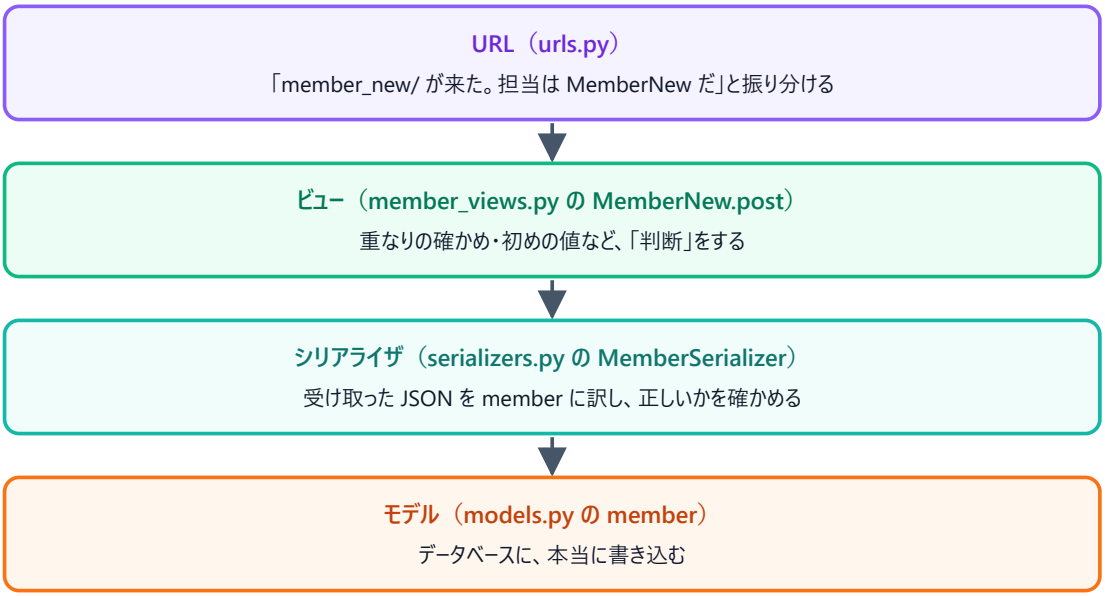


図7-2: 新しい人員を登録する時の、4 つの係の受け持ち。URL → ビュー → シリアライザ → モデルの順に渡していく

> **用語: シリアライズ（serialize）とデシリアライズ（deserialize）** シリアライズ＝「Pythonのオブジェクト → JSON文字列」への変換（データを送り出すとき）。デシリアライズ＝「JSON → Pythonオブジェクト」への逆変換（データを受け取るとき）。本アプリの `MemberSerializer` は、この両方向の翻訳を1つのクラスでこなします。

それでは、いよいよ実コードに入ります。まず HTTP の基本用語（§4）を押さえ、その後この章の主役 `member_views.py` を1行ずつ読んでいきましょう。

---

4. HTTPメソッドとステータスコード

コードを読む前に、何度も出てくる「HTTPメソッド」と「ステータスコード」を先に整理しておきます。

4.1 HTTPメソッド早見表

> **HTTPメソッド（method：メソッド）とは？** リクエストの「動詞」。「何をしたいのか」を表します。本アプリで使う主なものは次の4つ。

メソッド	意味	CRUD	本アプリの例
GET	取得（読む）	Read	人員一覧を表示 `MemberList.get`
POST	作成（送って作る）	Create	人員を新規登録 `MemberNew.post`
PUT	更新（置き換える）	Update	人員データを編集 `MemberUpdate.put`
DELETE	削除（消す）	Delete	人員を削除 `MemberDelete.delete`

> **CRUD（クラッド）とは？** Create（作る）・Read（読む）・Update（更新する）・Delete（消す）の頭文字。データを扱うアプリの基本4動作。`member_views.py` はこの4つがそのままクラスになっています。
>
> **なぜメソッドを分けるの？** 同じURLでも「読みたいのか・作りたいのか・消したいのか」で動作を変えたいからです。DRFは、来たメソッドに応じて自動で `get` / `post` / `put` / `delete` メソッドを呼び分けてくれます。プログラマは該当メソッドを書くだけでよいのです。

4.2 ステータスコード早見表

> **ステータスコード（status code：ステータスコード）とは？** 応答に付く3桁の番号。「うまくいったか・ダメだったか・なぜダメか」を表します。

番号	名前	意味	本アプリでの使用箇所
200	OK	成功	更新成功・一覧取得成功
201	Created	新規作成成功	人員の新規登録成功
202	Accepted	受理・処理中	非同期タスクがまだ処理中
204	No Content	成功・中身なし	人員の削除成功
400	Bad Request	送り手の入力ミス	番号重複・検査不合格
401	Unauthorized	未認証（ログインして）	未ログイン
403	Forbidden	権限不足	権限なしユーザー
404	Not Found	見つからない	対象データなし
500	Server Error	サーバー側の故障	プログラムの予期せぬエラー

> **番号の覚え方:** - **2xx** = 成功（やった！） - **4xx** = 送り手（クライアント）の問題（あなたのミス） - **5xx** = 受け手（サーバー）の問題（こちらの故障）

---

5. 実コード全解説：member_views.py 全225行

> このセクションが本章の心臓部です。`member_views.py` は **人員データの一覧・登録・更新・削除** を担当するAPIで、本アプリのCRUDの典型例です。ここを完全に理解すれば、他のビュー（kosu, team, def...）も同じパターンなので一気に読めるようになります。

5.1 冒頭の import（読み込み） — 1〜7行目

> **▼ このコードがやること（先に日本語で）**：このファイルが使う「外部の道具」を読み込みます。データベース操作の部品・DRFのAPI部品・このアプリ自身のモデルやシリアライザを、上から順番に取り寄せています。

コピー

```
# 条件付きの並べ替えに使う部品4種を読み込む
from django.db.models import Case, When, Value, IntegerField
# このアプリのmemberモデル（人員データの設計図）を読み込む
from ..models import member
# DRFのAPI用の基底クラス（土台）を読み込む
from rest_framework.views import APIView
# API応答を作るためのResponseを読み込む
from rest_framework.response import Response
# 200・404などのステータスコード集を読み込む
from rest_framework import status
# member⇔JSONの翻訳係を読み込む
from .serializers import MemberSerializer
# ページ送り（何件ずつ表示するか）の部品を読み込む
from ..utils.main_utils import CustomPagination
```

▶ ここで初めて出てくるコード

- **`Case`**（Django）：問い合わせの中で「場合分け」をする道具。`Case(When(条件, then=値), …, default=値)` で、条件ごとに違う値を作り、並べ替えなどに使います。
- **`When`**（Django）：`Case` の中の 1 つの場合。`When(shop='異動・退社', then=Value(1))` は「所属が『異動・退社』なら 1」。
- **`Value`**（Django）：問い合わせの中で使う「決まった値」。`Value(1)` のように書きます。
- **`IntegerField`**（Django）：`Case` などで作る値が「整数」だと Django に伝える物。`output_field=IntegerField()` と書きます。

1行ずつ意味を確認します。

- **1行目** `from django.db.models import Case, When, Value, IntegerField`
- `django.db.models` は「データベース操作の道具箱」。そこから4つの部品を取り出しています。
- `Case`・`When`・`Value` は **「もし〜なら、この値にする」という条件式** を、データベースの問い合わせの中で作るための部品です（後の§5.5で「異動・退社」を最後に並べるために使います）。
- `IntegerField`（整数フィールド）は「その結果は整数だよ」とデータ型を指定する部品です。

用語: import（インポート）「読み込む・取り寄せる」という意味。`from A import B` は「Aという場所からBという道具を取り寄せる」。料理でいえば「冷蔵庫（A）から卵（B）を取り出す」イメージ。これを書かないと、その道具は使えません。

- **2行目** `from ..models import member`
- `..` は「1つ上のフォルダ」を表します。`member_views.py` は `kosu/views/` の中にあるので、`..` は `kosu/` を指します。つまり `kosu/models.py` から `member` を読み込んでいます。

- `member` は人員1人を表すデータの設計図（モデル）。従業員番号・名前・所属・休憩時間などの欄を持ちます（中身は第8章）。

> ⚠️ **ドットの数に注意** `.` は「同じフォルダ」、`..` は「1つ上のフォルダ」。`from .serializers`（6行目）は同じ `views/` フォルダの `serializers.py`、`from ..models`（2行目）は1つ上の `kosu/models.py`。間違えると `ImportError`（読み込み失敗）になります。

- **3行目** `from rest_framework.views import APIView`
- DRFが用意した `APIView`（エーピーアイビュー）という土台のクラス。これを継承（後述）すると、`get`・`post` などのメソッドを書くだけでAPIが作れます。
- **4行目** `from rest_framework.response import Response`
- `Response`（レスポンス）は「APIの返事」を作るための部品。`Response({...}, status=...)` の形で、データと番号をセットで返します。
- **5行目** `from rest_framework import status`
- `status`（ステータス）は `HTTP_200_OK`・`HTTP_404_NOT_FOUND` のような **番号に名前を付けた一覧**。数字の `200` を直接書くより `status.HTTP_200_OK` と書くほうが意味が分かりやすいので使います。
- **6行目** `from .serializers import MemberSerializer`
- 同じ `views/` フォルダにある `serializers.py` から `MemberSerializer` を読み込みます。これがデータ⇔JSONの翻訳係（§6で詳説）。
- **7行目** `from ..utils.main_utils import CustomPagination`
- `kosu/utils/main_utils.py` から `CustomPagination`（カスタムページネーション）を読み込みます。一覧を「20件ずつ」のようにページ分けする部品（§8で詳説）。

> **用語: クラス（class）と継承（けいしょう）** クラスは「同じ性質を持つものの設計図」。`class MemberList(APIView):` は「`APIView` という親の設計図を引き継いで（継承して）、`MemberList` という子を作る」という意味。親が持つ機能（リクエストの受け取り方など）はそのまま使え、足りない部分（実際の処理）だけ自分で書けばよい、というのが継承の便利さです。家業を継ぐイメージ。

5.2 MemberList クラスの宣言と get の入口 — 12〜18行目

> **▼ このコードがやること（先に日本語で）**：「人員一覧」を返すAPIの本体を定義します。`get` メソッドは、ブラウザが「一覧をください（GET）」と言ってきたときに動く処理です。最初にセッション（ログイン情報の保管庫）から、ログイン中の従業員番号と、使う工数区分定義のバージョンを取り出します。

コピー

```
# 人員一覧
# APIViewを継承して「人員一覧API」クラスを作る
class MemberList(APIView):
  # GET時の動作
  # GETリクエストが来たら呼ばれるメソッド
  def get(self, request):
    # セッションからデータ取得
    # セッションからログイン中の従業員番号を取り出す
    login_no = request.session.get('login_No')
    # セッションから使用中の工数区分定義バージョンを取り出す
    def_ver = request.session.get('input_def')
```

- **13行目** `class MemberList(APIView):`
- `APIView` を継承した `MemberList` クラスを宣言。このクラス1つで「人員一覧」というAPIの窓口になります。
- **15行目** `def get(self, request):`
- `def`（デフ）は「関数（処理のかたまり）を定義する」キーワード。
- メソッド名が `get` なので、これは **GETリクエスト**（＝「データをください」という読み取りのお願い）が来たときに自動で呼ばれます。DRFが「GETならgetメソッドを呼ぶ」と振り分けてくれます。
- `self`（セルフ）は「このクラス自身」を指す決まりの第1引数（とりあえず「おまじない」と思ってよい）。
- `request`（リクエスト）には、ブラウザから届いたお願いの全情報（誰が・何を・どんな付帯情報で）が入っています。

> **用語: メソッド（method）** クラスの中に書かれた関数のこと。`MemberList` というクラスの中の `get` は「MemberListのgetメソッド」と呼びます。クラスが持つ「動作」だと思ってください。

- **17行目** `login_no = request.session.get('login_No')`
- `request.session` は **セッション**（後述）という「サーバー側の一時保管庫」。ログイン時にここへ従業員番号を保存しておき、以降のリクエストで取り出します。
- `.get('login_No')` は「`login_No` という名札の値を取り出す」。もし無ければ `None`（なし）が返ります。
- 取り出した値を `login_no` という変数に入れています。

> **用語: セッション（session：セッション）** ブラウザとサーバーの「会話の記憶」。HTTPは本来「1回ごとに忘れる」性質（毎回はじめまして）なので、ログイン状態を覚えておくためにセッションを使います。ログイン時に「この人は従業員番号123番」とサーバー側に記録し、ブラウザには「あなたの記憶の鍵（クッキー）」だけを渡します。次回以降、鍵を見せれば「ああ123番さんね」と思い出せる、という仕組み。本アプリでは `SESSION_ENGINE` でデータベースに記憶しています（§12）。

- **18行目** `def_ver = request.session.get('input_def')`

- 同様に、セッションから「使用中の工数区分定義バージョン」を取り出して `def_ver` に入れます。

5.3 ログイン・権限チェック — 20〜34行目

> **▼ このコードがやること（先に日本語で）**：この一覧を見せてよい相手かを順番に確認します。①ログインしていない人を弾く ②定義バージョンが無い人を弾く ③その従業員に「管理権限」があるか確認し、なければ弾く ④そもそも人員データが見つからなければ弾く。問題があれば、その場で適切なエラー番号を付けて返し、処理を中断します。

コピー

```
  # 未ログインや定義が未定義の場合はログイン画面へ
  # ログイン番号が無い（未ログイン）なら
  if not login_no:
    # 401を返して中断
    return Response({'status': 'error', 'message': 'ログイン情報が確認できません。'}, status=status.HTTP_401_UNAUTHORIZED)
  # 定義バージョンが無いなら
  if not def_ver:
    # 401を返して中断
    return Response({'status': 'error', 'message': '使用する工数区分定義情報が確認できません。'}, status=status.HTTP_401_UNAUTHORIZED)

  try:
    # ログインユーザーのデータ取得
    # 従業員番号でログイン者の人員データを1件取得
    member_data = member.objects.get(employee_no=login_no)
    # 権限がない場合はMenu画面へ
    # その人に管理権限が無いなら
    if not member_data.authority:
      # 403を返して中断
      return Response({'status': 'error', 'message': 'アクセス権限がありません。'}, status=status.HTTP_403_FORBIDDEN)
  # 該当人員が見つからなかった場合
  except member.DoesNotExist:
    # 人員情報取得できない場合エラー
    # 401を返して中断
    return Response({'status': 'error', 'message': '人員情報が見つかりません。'}, status=status.HTTP_401_UNAUTHORIZED)
```

- **21行目** `if not login_no:`
- `not login_no` は「`login_no` が空（None や空文字）なら True」。つまり「未ログインなら」という条件。
- **22行目** `return Response({...}, status=status.HTTP_401_UNAUTHORIZED)`
- `return`（リターン）でその場で処理を打ち切り、返事を返します。これより下の行は実行されません。
- 返すデータは辞書 `{'status': 'error', 'message': '...'}`（後でJSONになる）。
- `status=status.HTTP_401_UNAUTHORIZED` は **401（未認証）**。「あなたが誰か分からない＝ログインして」という意味の番号です。

> **用語: 辞書（dictionary：ディクショナリ）** `{'名前': 値, '名前': 値}` の形で「名札と値のペア」を並べたPythonのデータ型。JSONとそっくりで、DRFはこれを自動でJSONに変換します。

- **23〜24行目** 同じ形で、定義バージョンが無ければ401で中断します。
- **26行目** `try:`
- `try`（トライ）は「以下を試してみる。途中でエラーが起きたら、対応する `except` に飛ぶ」という構文。データベース問い合わせは「見つからない」エラーが起きうるので、安全に囲みます。

> **用語: try / except（トライ・エクセプト）**「やってみて（try）、ダメだったら（except）こうする」という例外処理。料理でいえば「卵を割ってみて（try）、もし腐っていたら（except）別の卵を使う」。エラーでプログラム全体が止まるのを防ぎます。

- **28行目** `member_data = member.objects.get(employee_no=login_no)`
- `member.objects` は「memberテーブルへの入口」（マネージャと呼ぶ）。
- `.get(employee_no=login_no)` は「従業員番号が `login_no` と一致する人員を **1件だけ** 取ってくる」。
- 見つかれば、その人員データが `member_data` に入ります。見つからなければ `member.DoesNotExist` というエラーが発生し、32行目の `except` に飛びます。

> **用語: ORM（オーアールエム）／クエリ** ORM（Object-Relational Mapping）は「Pythonの書き方で、データベースを操作できる仕組み」。`member.objects.get(...)` のように書くと、Djangoが裏で `SELECT * FROM member WHERE ...` というSQL（データベースの命令文）に翻訳してくれます。SQLを直接書かなくていいのがORMの利点。`クエリ` はこの「データベースへの問い合わせ」のこと。

- **30行目** `if not member_data.authority:`
- `member_data.authority` は、その人員の「管理権限フラグ」（True/Falseの欄）。`not` を付けているので「権限が無いなら True」。
- **31行目** 権限が無ければ **403（禁止）** を返します。「あなたが誰かは分かったが、この操作は許可されていない」という意味の番号です。

> ⚠️ **401と403の違い** 401は「あなたが誰か分からない（ログインして）」。403は「あなたが誰かは分かったが、権限が足りない」。本アプリでは、未ログイン＝401、ログイン済みだが権限なし＝403、と使い分けています。

- **32行目** `except member.DoesNotExist:`
- 28行目で人員が見つからなかったときに飛んでくる先。`member.DoesNotExist` は「該当レコードなし」を表す専用のエラー。
- **34行目** 人員が見つからなければ401を返します。

5.4 検索条件の取得 — 36〜38行目

> **▼ このコードがやること（先に日本語で）**：URLの末尾に付いてくる「絞り込み条件」を取り出します。たとえば `?employee_no=123&shop=W1` のように付いてくる検索キーワードを読み取ります。

コピー

```
    # クエリパラメータで絞り込み条件を取得
    # URLの?employee_no= の値を取り出す（無ければNone）
    search_number = request.query_params.get('employee_no', None)
    # URLの?shop= の値を取り出す（無ければNone）
    search_shop = request.query_params.get('shop', None)
```

- **37行目** `request.query_params.get('employee_no', None)`
- `query_params`（クエリパラメータ）は、URLの `?` 以降に付いてくる検索条件。たとえば `/api/member_list/?employee_no=123` なら、`employee_no` の値は `'123'`。
- `.get('employee_no', None)` は「`employee_no` があれば取り出す、無ければ `None`」。第2引数の `None` が「見つからなかったときの値」です。

> **用語: クエリパラメータ（query parameter：クエリパラメータ）** URLの `?` 以降に `名前=値&名前=値` の形で付ける、検索・絞り込みのための付帯情報。`http://例/member_list/?shop=W1` の `shop=W1` の部分。「絞り込みの注文票」だと思ってください。

- **38行目** 同様に、所属（shop）での絞り込み条件を取り出します。

5.5 全人員を「異動・退社を最後に」並べ替えて取得 — 40〜47行目

> **▼ このコードがやること（先に日本語で）**：全人員データを取り出すのですが、そのとき「**異動・退社**」になった人を**一覧の最後に回す** ための工夫をしています。一時的に「shop_order」という並べ替え用の隠し番号を各人に付け（異動・退社なら1、それ以外は0）、まずその番号順、次に従業員番号順に並べます。

コピー

```
    # 人員データ全取得
    # 各人員に計算結果の列を一時的に追加する
    members = member.objects.annotate(
        # shop_orderという並べ替え用の値を作る
        shop_order=Case(
            # もしshopが「異動・退社」なら1にする
            When(shop='異動・退社', then=Value(1)),
            # それ以外は0にする
            default=Value(0),
            # この値は整数型だと指定する
            output_field=IntegerField(),
        )
    # shop_order昇順→従業員番号昇順で並べる
    ).order_by('shop_order', 'employee_no')
```

▶ ここで初めて出てくるコード

- **`.annotate()`**（Django）：問い合わせの 1 行ごとに、計算した値を付け足すメソッド。`member.objects.annotate(shop_order=Case(…))` は、人員の 1 人ずつに「並べ替え用の番号（ `shop_order` ）」を付けます。

ここは少し難しいので、丁寧に分解します。

- **41行目** `members = member.objects.annotate(`
- `annotate`（アノテート＝注釈を付ける）は「各レコードに、計算した値を一時的な列として追加する」メソッド。ここでは `shop_order` という新しい列を各人員に付け足します。

用語: annotate（アノテート）「注釈・付箋を付ける」。データベースから取ってくる各行に、その場限りの計算列を貼り付けるイメージ。実際のテーブルには列は増えません。ここでは「並べ替え用の隠し番号」を一時的に付けています。

- **42〜46行目** `Case(...)` の中身
- `Case`（ケース）と `When`（ウェン）は「もし〜なら〜」という条件分岐をデータベース側で作る部品。
- **43行目** `When(shop='異動・退社', then=Value(1))` …「もし所属が『異動・退社』なら、値を `1` にする」。
- **44行目** `default=Value(0)` …「上の条件に当てはまらなければ、値は `0`」。
- **45行目** `output_field=IntegerField()` …「この `shop_order` という列は整数型ですよ」とDjangoに教えます。
- **47行目** `.order_by('shop_order', 'employee_no')`
- `order_by`（オーダーバイ）は「並べ替え」。
- 第1キー `shop_order` で並べると、`0`（現役）が先、`1`（異動・退社）が後ろになります。
- 同じ `shop_order` の中では、第2キー `employee_no`（従業員番号）の小さい順に並びます。

なぜこんな工夫をするの？ 単純に従業員番号順に並べると、退社した人が現役の人に混ざってしまい見づらいからです。そこで「異動・退社」だけ一覧の最後にまとめる。直近のコミット「人員一覧で異動・退社を後ろに変更」がまさにこの処理です。

⚠️ **この時点ではまだデータベースに問い合わせていない** `members` には「こういう問い合わせをするよ」という **設計図（クエリセット）** が入っているだけで、実際のデータ取得はまだ起きていません。Djangoは「本当に必要になった瞬間」まで問い合わせを遅らせます（遅延評価）。実際にデータが取られるのは、後のページネーション（§5.7）の段階です。

5.6 絞り込み（フィルタリング） — 49〜53行目

▼ このコードがやること（先に日本語で）：§5.4で取り出した検索条件があれば、それに合うものだけに絞り込みます。従業員番号は「部分一致」（含んでいればOK）、所属は「完全一致」で絞ります。

コピー

```
    # 絞り込みある場合はフィルタリング
    # 従業員番号の検索条件があれば
    if search_number:
      # 番号を「含む」もので絞り込む
      members = members.filter(employee_no__icontains=search_number)
    # 所属の検索条件があれば
    if search_shop:
      # 所属が完全一致するもので絞り込む
      members = members.filter(shop=search_shop)
```

▶ ここで初めて出てくるコード

- **`列__条件=`**（Django）: Django の ORM の `filter(work_day2__gte=start_day, work_day2__lte=end_day)` のように、列の名前の後ろに `__` と条件を付けると、いろいろな比べ方ができます。`__gte`（以上）・`__lte`（以下）・`__gt`（より大きい）・`__in`（並びのどれか）・`__icontains`（大文字・小文字を区別せず、含む）・`__isnull`（空か）など。

- **50行目** `if search_number:` … 検索番号が入力されていれば（空でなければ）True。
- **51行目** `members.filter(employee_no__icontains=search_number)`
- `filter`（フィルター）は「条件に合うものだけ残す」。
- `employee_no__icontains` の `__icontains`（アイコンテインズ）は「**大文字小文字を区別せず、含む**」という条件。たとえば検索が `12` なら、`12`・`123`・`512` などすべてヒットします。

用語: フィールドルックアップ（`__oo`） Djangoでは、フィールド名のうしろに `__` を付けて条件の種類を指定します。`__icontains`（含む）・`__gte`（以上）・`__lte`（以下）・`__exact`（完全一致）など。アンダースコア2個がポイントです。

- **52〜53行目** 所属の条件があれば、`shop=search_shop` で **完全一致** 絞り込み。番号と違い、所属はピッタリ一致を求めます。

なぜ番号は部分一致、所属は完全一致なの？ 番号は「12と入れたら12番台を探したい」という曖昧検索が便利。一方、所属は「W1」「W2」など決まった値なので、ピッタリ一致のほうが正確だからです。

5.7 ページネーションと応答 — 55〜60行目

▼ このコードがやること（先に日本語で）：絞り込んだ全データを一度に全部返すと重いので、「今のページ分」だけ切り出します（ページ送り）。切り出したデータをJSON形式に翻訳し、「全部で何件あるか」などの情報と一緒に返します。

コピー

```
    # ページネーション処理
    # ページ送り係を1つ用意する
    paginator = CustomPagination()
    # 全データから「今のページ分」だけ切り出す
    result_page = paginator.paginate_queryset(members, request)
    # 切り出した複数件をJSON用に翻訳する
    serializer = MemberSerializer(result_page, many=True)

    # 件数情報付きでJSONを返す
    return paginator.get_paginated_response(serializer.data)
```

▶ ここで初めて出てくるコード

- **`.paginate_queryset()`**（Django REST framework）：`paginator.paginate_queryset(一覧, request)` は、リクエストの `?page=2` などを見て、そのページの分だけを取り出します。

- **56行目** `paginator = CustomPagination()`
- §8で詳しく見る `CustomPagination` を1つ作ります。この瞬間に「1ページ何件表示するか」が、管理者設定から動的に決まります。
- **57行目** `result_page = paginator.paginate_queryset(members, request)`
- `paginate_queryset`（ページネイト・クエリセット）は「全データ `members` から、`request` が要求するページ番号の分だけ切り出す」メソッド。
- たとえば1ページ20件で2ページ目を要求していれば、21〜40件目を返します。**ここで初めて実際にデータベースに問い合わせが走ります。**
- **58行目** `serializer = MemberSerializer(result_page, many=True)`
- 切り出した人員データ（複数件）を `MemberSerializer` に渡して、JSONに翻訳する準備をします。
- `many=True`（メニー・トゥルー）は「対象は1件ではなく **複数件** だよ」という指定。1件のときは付けません。
- **60行目** `return paginator.get_paginated_response(serializer.data)`
- `serializer.data` が、翻訳済みのJSONデータ（人員のリスト）。

- `get_paginated_response` が、それに「全部で何件か（count）」「1ページ何件か（page_size）」などを足して、最終的な応答を組み立てます（中身は§8）。

> **▼ こう表示されれば成功:** React側がこのAPIを呼ぶと、次のようなJSONが返ってきます（※説明用の簡易例）。

コピー

```
{
  "count": 53,
  "page_size": 20,
  "results": [
    {"employee_no": 101, "name": "山田太郎", "shop": "W1", ...},
    {"employee_no": 102, "name": "佐藤花子", "shop": "W1", ...}
  ]
}
```

`count` が全件数、`results` が今のページ分。Reactはこれを表に並べ、ページ番号を作ります。

5.8 MemberNew クラス（新規登録）の get — 65〜82行目

> **▼ このコードがやること（先に日本語で）:** 「新規登録」ページを開いたとき、登録フォームの初期表示用にログイン者自身のデータを返します。ここでも先に権限チェックを行い、権限が無ければ弾きます。

コピー

```
# 人員データ新規登録
# 新規登録API
class MemberNew(APIView):
  # 登録画面を開いたときのGET
  def get(self, request):
    # セッションからログイン番号を取得
    login_no = request.session.get('login_No')
    # 未ログインなら
    if not login_no:
      # 401で中断
      return Response({'status': 'error', 'message': 'ログイン情報が確認できません。'}, status=status.HTTP_401_UNAUTHORIZED)

    try:
      # ログインユーザーのデータ取得
      # ログイン者の人員データ取得
      member_data = member.objects.get(employee_no=login_no)
      # 権限がない場合はMenu画面へ
      # 権限が無ければ
      if not member_data.authority:
        # 403で中断
        return Response({'status': 'error', 'message': 'アクセス権限がありません。'}, status=status.HTTP_403_FORBIDDEN)
    # 人員が見つからなければ
    except member.DoesNotExist:
      # 人員情報取得できない場合エラー
      # 401で中断
      return Response({'status': 'error', 'message': '人員情報が見つかりません。'}, status=status.HTTP_401_UNAUTHORIZED)

    # ログイン者データをJSON化
    serializer = MemberSerializer([member_data], many=True)
    # そのまま返す
    return Response(serializer.data)
```

- **65行目** `class MemberNew(APIView):` … 新規登録を担当するクラス。

- **66〜79行目** … §5.3とほぼ同じ「ログイン確認→権限確認」の定番チェック。本アプリの全ビューに共通する「門番」です。
- **81行目** `serializer = MemberSerializer([member_data], many=True)`
- `[member_data]` と角カッコで囲って **1件をリストにして** 渡し、`many=True` を付けています。Reactが「常にリストで受け取る」前提なので、1件でもリスト形式に揃えています。
- **82行目** `return Response(serializer.data)`
- ステータスコードを指定していません。この場合、DRFは自動で **200（成功）** を付けます。

5.9 MemberNew の post（新規登録の本体） — 85〜151行目

ここがCRUDの「C（Create＝作る）」の本体です。長いので、ブロックに分けて読みます。

5.9.1 受信データの取り出しと重複チェック — 85〜89行目

> **▼ このコードがやること（先に日本語で）**：Reactから送られてきた登録データを受け取り、まず「その従業員番号がすでに登録済みでないか」を確認します。すでにあれば、エラー（400）を返して登録を止めます。

コピー

```
# 登録ボタンが押されたときのPOST
def post(self, request):
  # 送られてきた登録データを取り出す
  data = request.data

  # 同じ従業員番号が既に存在するか確認
  if member.objects.filter(employee_no=data.get('employee_no')).exists():
    # あれば400で中断
    return Response({'status': 'error', 'message': '入力した従業員番号はすでに登録されています。'}, status=status.HTTP_400_BAD_REQUEST)
```

- **85行目** `def post(self, request):`
- メソッド名が `post` なので、**POSTリクエスト**（＝「データを送るので保存して」という書き込みのお願い）が来たときに呼ばれます。

> **用語: POST（ポスト）**「データを送って新しく作る」ためのHTTPメソッド。郵便ポストに手紙を投函するイメージ。GETが「読み取り」なのに対し、POSTは「新規作成」。

- **86行目** `data = request.data`
- `request.data` には、Reactが送ってきた登録フォームの中身（JSON）が、Pythonの辞書として入っています。
- **88行目** `if member.objects.filter(employee_no=data.get('employee_no')).exists():`
- `data.get('employee_no')` で、送られてきた従業員番号を取り出します。
- `member.objects.filter(employee_no=...)` で「その番号の人員」を絞り込み、`.exists()` で「1件でも存在するか？」を True/False で確認します。
- `.exists()` は実際のデータを取らず「あるかないか」だけ調べるので軽い、というのもポイント。

- **89行目** すでに同じ番号があれば、**400（不正なリクエスト）** を返して中断します。

> **用語: 400 Bad Request（バッドリクエスト）**「あなたの送ってきた内容に問題があります」を表す番号。入力ミス・重複・形式違反などはこれ。サーバー側の故障（500番台）ではなく、**送り手（クライアント）側の問題** を表します。

5.9.2 休憩時間の初期値設定 — 91〜144行目

> **▼ このコードがやること（先に日本語で）**：新しい人員には、所属に応じた「標準の休憩時間」をあらかじめセットします。所属が「現場系（W1, W2, A1, A2, J, 組長以上）」なら現場用の時間割を、それ以外（事務系など）なら別の時間割を、24個の休憩欄に一括で書き込みます。これにより、登録者が毎回休憩を手入力しなくて済みます。

なぞってコピーすると「…」が別の文字になります。正しくは → コピー

```
    # 所属が現場系グループに含まれるか判定
    if data.get('shop') in ['W1', 'W2', 'A1', 'A2', 'J', '組長以上(W,A)']:
      # dataに休憩時間の初期値を一括で上書き追加
      data.update({
        # 休憩1（例：11:40〜12:40）
        'break_time1': '#11401240',
        # 休憩1の残業時パターン1
        'break_time1_over1': '#17201735',
        # 休憩1の残業時パターン2
        'break_time1_over2': '#23350035',
        # 休憩1の残業時パターン3
        'break_time1_over3': '#04350450',
        # 休憩2
        'break_time2': '#14101510',
        # …（中略：break_time2_over1 〜 break_time6_over3 まで同形式で続く）…
        # 休憩6の残業時パターン3
        'break_time6_over3': '#12201230',
      })
    # 上記グループ以外（事務系など）の場合
    else:
      # 別の休憩時間パターンを一括設定
      data.update({
        # 休憩1（事務系の標準）
        'break_time1': '#10401130',
        # …（中略：以下同様に24欄ぶん設定）…
        # 休憩6の残業時パターン3
        'break_time6_over3': '#12201230',
      })
```

▶ ここで初めて出てくるコード

- `.update()`（Python）：辞書に、別の辞書の中身をまとめて入れる（上書きする）メソッド。Django の問い合わせの `.update(項目=値)` は、合う行をまとめて書きかえる、別の意味です。

⚠️ 上のコードは紙面の都合で中略表示しています。実ファイル（91〜144行目）では、`if` 側・`else` 側ともに**24個の休憩欄（break_time1〜6 と、それぞれの over1〜3）**がすべて明示的に書かれています。中略部分も同じ `'欄名': '#値'` の形式が並んでいるだけで、構造は変わりません。

- **91行目** `if data.get('shop') in ['W1', 'W2', 'A1', 'A2', 'J', '組長以上(W,A)']:`
- `in [...]` は「その値がリストの中に含まれるか」を判定します。所属が現場系の6種のどれかなら True。
- **92行目** `data.update({...})`
- `update`（アップデート）は、辞書 `data` に新しいキーと値をまとめて追加・上書きするメソッド。ここでは24個の休憩欄を一気にセットしています。
- **各値の意味** … `'#11401240'` のような文字列は、本アプリ独自の時間表現です。`#` の後ろを4桁ずつ区切って「`1140` ＝11時40分」「`1240` ＝12時40分」と読み、「11:40〜12:40が休憩」を表します。`_over1` 〜 `_over3` は、残業の有無で休憩時間がずれるパターン違いです（詳しい意味は工数計算の章で扱います）。
- **118行目** `else:`
- 所属が現場系グループ **以外** の場合に実行される側。こちらも同様に24欄を、別の値でセットします。

なぜこんなに初期値を持たせるの？ 休憩時間は、現場と事務で時間割が違い、しかも残業の有無で変わる複雑なものです。新規登録のたびに24個も手入力させるとミスが起きるので、所属を選べば標準値が自動で入るようにしている、という親切設計です。後で個別に変更することもできます。

⚠️ **`if` と `else` の重複に注意** よく見ると、`break_time4` 〜 `break_time6` 系の値は `if` 側と `else` 側で同じです。違うのは `break_time1` 〜 `break_time3` 系だけ。現場系と事務系で午前の休憩割が異なる、という業務ルールを反映しています。コードを直すときは、両方のブロックを揃えて直す必要があるので注意してください。

5.9.3 検証・保存・応答 — 146〜151行目

▼ このコードがやること（先に日本語で）：完成したデータをシリアライザに渡して「中身が正しいか」を検査します。検査に合格すればデータベースに保存し、201（作成成功）を返します。不合格なら400（入力エラー）を返します。

コピー

```
    # 送信データを翻訳・検査の準備
    serializer = MemberSerializer(data=data)
    # 検査に合格したら
    if serializer.is_valid():
      # データベースに新規保存
      serializer.save()
      # 保存した内容を201で返す
      return Response(serializer.data, status=status.HTTP_201_CREATED)

    # 検査不合格なら400で返す
    return Response({'status': 'error', 'message': 'バリテーションエラー'}, status=status.HTTP_400_BAD_REQUEST)
```

▶ ここで初めて出てくるコード

- `.is_valid()`（Django REST framework）：送られてきたデータが、シリアライザー（入力の確かめ係）の決まりに合うかを調べるメソッド。`raise_exception=True` で、合わない時は 400 の答えにします。

- **146行目** `serializer = MemberSerializer(data=data)`
- `data=data` のように **キーワード** `data=` **を付けて** 渡すのがポイント。これは「**受信したデータを検査・保存したい（書き込みモード）**」の合図です。
- （対して§5.7のように `MemberSerializer(result_page)` と渡すと「**取得したオブジェクトをJSONにしたい（読み出しモード）**」になります。同じシリアライザが、渡し方で役割を変えるのです。）
- **147行目** `if serializer.is_valid():`
- `is_valid`（イズ・バリッド＝妥当か）は「送られたデータがモデルの決まり（型・必須・桁数など）を満たしているか」を検査するメソッド。合格なら True。

用語: バリデーション（validation：バリデーション）「入力の検証・妥当性チェック」。たとえば「従業員番号は数字か」「必須欄が埋まっているか」を確認すること。不正なデータがデータベースに入るのを防ぐ「門番」の役目。`is_valid()` がこれを担います。なお151行目のメッセージは「バリテーション」と書かれていますが、これは本アプリ内の表記（タイプミス）で、動作には影響しません。

- **148行目** `serializer.save()`
- 検査に合格したデータを、**実際にデータベースに書き込みます**。新規なので新しい行が1つ増えます。
- ⚠️ この `save()` の瞬間に、§9で学ぶ **signals（シグナル）が自動で発火** し、「CREATE（新規作成）」の履歴が `History` テーブルに残ります。ビューのコードには履歴処理が一切書かれていないのに、自動で記録される——これがシグナルの魔法です。
- **149行目** `return Response(serializer.data, status=status.HTTP_201_CREATED)`
- 保存した内容を、**201（作成成功）** を付けて返します。

> **用語: 201 Created（クリエイテッド）**「新しいデータの作成に成功した」を表す番号。単なる成功の200より具体的で、「新規作成が成功した」ことをはっきり示します。

- **151行目** `is_valid()` が False（検査不合格）だった場合、ここまで来て400を返します。

5.10 MemberUpdate クラス（更新） — 156〜203行目

CRUDの「U（Update＝更新）」を担当します。

5.10.1 get_object ヘルパー — 157〜161行目

> **▼ このコードがやること（先に日本語で）:**「指定された従業員番号の人員を1件取ってくる」共通処理を、小さな部品（メソッド）として切り出しています。見つからなければ `None` を返します。更新と削除で何度も使うので、まとめておくと便利です。

コピー

```
# 人員データ編集動作
# 更新API
class MemberUpdate(APIView):
  # 主キー(pk)で1件取ってくる共通メソッド
  def get_object(self, pk):
    try:
      # 従業員番号がpkの人員を返す
      return member.objects.get(employee_no=pk)
    # 見つからなければ
    except member.DoesNotExist:
      # Noneを返す
      return None
```

- **157行目** `def get_object(self, pk):`
- `pk`（ピーケー）は **Primary Key（プライマリキー＝主キー）** の略。「1件を特定するための鍵」。ここでは従業員番号が鍵の役割です。`pk` はURLから渡されます（§7参照）。

> **用語: 主キー（primary key：プライマリキー）** テーブルの中で、1行を一意に特定するための「背番号」。同じ値は2つと存在しません。図書館の本に貼られたバーコードのようなもの。これがあれば「どの本（行）か」を間違いなく指定できます。

- **159行目** `return member.objects.get(employee_no=pk)` … 従業員番号が `pk` の人員を取得して返します。
- **160〜161行目** 見つからなければ `None`（なし）を返します。呼び出し側で「Noneなら無かった」と判断できます。

5.10.2 update の get（編集画面の初期表示）— 164〜185行目

> **▼ このコードがやること（先に日本語で）**：編集画面を開いたとき、編集対象の人員データを返します。ここでも「ログイン確認→権限確認→対象が存在するか確認」の順で門番チェックを行います。

コピー

```
# 編集画面を開いたときのGET（pk付き）
def get(self, request, pk):
  # セッションからログイン情報を取得
  # ログイン番号取得
  login_no = request.session.get('login_No')
  # 未ログインなら
  if not login_no:
    # 401
    return Response({'status': 'error', 'message': 'ログイン情報が確認できません。'}, status=status.HTTP_401_UNAUTHORIZED)

  # ログインユーザーの権限を確認
  try:
    # ログイン者データ取得
    member_data = member.objects.get(employee_no=login_no)
  # 見つからなければ
  except member.DoesNotExist:
    # 403
    return Response({'status': 'error', 'message': '権限確認中にエラーが発生しました'}, status=status.HTTP_403_FORBIDDEN)
  # 権限が無ければ
  if not member_data.authority:
    # 403
    return Response({'status': 'error', 'message': 'アクセス権限がありません。'}, status=status.HTTP_403_FORBIDDEN)

  # 指定された従業員データを取得
  # 編集対象を取得（共通メソッド利用）
  member_instance = self.get_object(pk)
  # 対象が無ければ
  if member_instance is None:
    # 404
    return Response({'status': 'error', 'message': '人員データが確認できません。'}, status=status.HTTP_404_NOT_FOUND)

  # データをシリアライズして返却
  # 1件をJSON化（many不要）
  serializer = MemberSerializer(member_instance)
  # 返す
  return Response(serializer.data)
```

- **164行目** `def get(self, request, pk):`
- §5.2の `get` と違い、引数に `pk` が増えています。URLに `member_update/123/` のように番号が含まれるので、その `123` が `pk` として渡ってきます（§7で配線を確認します）。
- **166〜176行目** … おなじみの門番チェック。ログイン確認と権限確認です。
- **179行目** `member_instance = self.get_object(pk)`
- `self.get_object(pk)` で、§5.10.1の共通メソッドを呼び、編集対象を取得します。`self.` は「このクラス自身のメソッドを呼ぶ」という書き方。
- **180〜181行目** 対象が `None`（見つからない）なら **404（見つからない）** を返します。

> **用語: 404 Not Found（ノットファウンド）**「探したけれど、そのデータが見つからない」を表す番号。Webで「ページが見つかりません」と出るあの404です。

- **184行目** `serializer = MemberSerializer(member_instance)`
- 1件だけなので `many=True` は付けません。1件のオブジェクトをそのままJSONに翻訳します。

5.10.3 update の put（更新の本体） — 188〜203行目

> **▼ このコードがやること（先に日本語で）**：編集内容を受け取り、対象が存在するか確認し、検査に合格したら保存します。さらに「従業員番号を別の既存番号に変えようとしていないか」も二重チェックして、番号の重複を防ぎます。

長い行を折り返しているので、なぞってコピーすると行が分かれます。正しくは → コピー

```
  # 更新ボタンが押されたときのPUT
  def put(self, request, pk):
    # 特定の従業員データを取得
    # 更新対象を取得
    member_instance = self.get_object(pk)
    # 対象が無ければ
    if member_instance is None:
      # 404
      return Response({'status': 'error', 'message': 'レコードが見つかりません。'},
status=status.HTTP_404_NOT_FOUND)

    # クライアントから送られてきたデータをシリアライズ
    # 送られた更新データ取得
    data = request.data
    # 既存インスタンス＋新データで更新準備
    serializer = MemberSerializer(member_instance, data=data)
    # 検査に合格したら
    if serializer.is_valid():
      # 従業員番号の一意性確認
      # 番号を変更し、その番号が既に他にあれば
      if data.get('employee_no') != pk and
member.objects.filter(employee_no=data.get('employee_no')).exists():
        # 400
        return Response({'status': 'error', 'message': '入力した従業員番号はすでに登録されていま
す。'},status=status.HTTP_400_BAD_REQUEST)
      # 更新を保存
      serializer.save()
      # 200で返す
      return Response(serializer.data, status=status.HTTP_200_OK)
    # 検査不合格なら400
    return Response({'status': 'error', 'message': 'バリテーションエラー'},
status=status.HTTP_400_BAD_REQUEST)
```

- **188行目** `def put(self, request, pk):`
- メソッド名が `put` なので、**PUTリクエスト**（＝「既存データを丸ごと差し替えて」という更新のお願い）が来たときに呼ばれます。

> **用語: PUT（プット）**「既存のデータを更新（置き換え）する」ためのHTTPメソッド。POSTが「新規作成」なのに対し、PUTは「既にあるものを書き換える」。

- **190〜192行目** 更新対象を取得し、無ければ404。

- **196行目** `serializer = MemberSerializer(member_instance, data=data)`
- ここが新規(POST)との違い。**第1引数に既存の** `member_instance`、`data=` **に新データ** を渡しています。
- これは「**この既存レコードを、新しいデータで更新する**」という更新モードの合図です。第1引数を渡さず `data=` だけなら新規、第1引数も渡せば更新、と覚えてください。
- **197行目** `if serializer.is_valid():` … データの妥当性検査。合格なら次へ。
- **199行目** `if data.get('employee_no') != pk and member.objects.filter(employee_no=data.get('employee_no')).exists():`
- 2つの条件を `and`（かつ）で繋いでいます。
- 左 `data.get('employee_no') != pk` …「従業員番号を **変更しようとしている**（元の `pk` と違う）」か。
- 右 `member.objects.filter(...).exists()` …「その新しい番号が **すでに他の人に使われている**」か。
- 両方が True、つまり「番号を変えようとしていて、かつその番号が既に存在する」場合だけ、200行目で400を返します。

> **なぜこの二重チェックが必要なの？** 番号を変えない更新（名前だけ直すなど）では、自分自身の番号は当然存在しているので、左の条件 `!= pk` で「変えていない」を先に判定して除外しています。これを忘れると「自分の番号と重複している」と誤判定して、普通の更新ができなくなってしまいます。

- **201行目** `serializer.save()` … 更新を保存。この瞬間も signals が発火し、「UPDATE（更新）」の履歴（変更前後の差分つき）が残ります。
- **202行目** 成功したら **200（成功）** を返します。新規の201と違い、更新は200を使います。

5.11 MemberDelete クラス（削除） — 208〜224行目

CRUDの「D（Delete＝消す）」を担当します。

> **▼ このコードがやること（先に日本語で）**：削除対象の人員を取得し、存在すれば削除します。存在しなければ404を返します。削除成功時は「中身のない成功（204）」を返します。

コピー

```
# 人員データ削除動作
# 削除API
class MemberDelete(APIView):
  # 1件取得の共通メソッド（更新と同じ形）
  def get_object(self, pk):
    try:
      # pkの人員を返す
      return member.objects.get(employee_no=pk)
    # 無ければ
    except member.DoesNotExist:
      # None
      return None


  # 削除ボタンが押されたときのDELETE
  def delete(self, request, pk):
    # 削除対象のオブジェクトを取得
    # 削除対象を取得
    member_instance = self.get_object(pk)
    # 無ければ
    if member_instance is None:
      # 404
      return Response({'status': 'error', 'message': 'レコードが見つかりません。'}, status=status.HTTP_404_NOT_FOUND)

    # レコードを削除
    # データベースから削除
    member_instance.delete()
    # 204で返す
    return Response({'status': 'success', 'message': 'レコードを削除しました。'}, status=status.HTTP_204_NO_CONTENT)
```

- **216行目** `def delete(self, request, pk):`
- メソッド名が `delete` なので、**DELETEリクエスト**（＝「このデータを消して」というお願い）が来たときに呼ばれます。

> **用語: DELETE（デリート）**「指定したデータを削除する」ためのHTTPメソッド。名前のとおり「消す」。

- **218〜220行目** 削除対象を取得し、無ければ404。
- **223行目** `member_instance.delete()`
- データベースから、その人員の行を **完全に削除** します。この瞬間も signals が発火し、「DELETE（削除）」の履歴が残ります（消す前に記録されるので、後から「誰が何を消したか」を追えます）。
- **224行目** `return Response({...}, status=status.HTTP_204_NO_CONTENT)`
- **204（内容なし）** を返します。「成功したが、返す中身は無い」という意味。削除では「消したので、もう返すデータがない」ため204が定番です。

> **用語: 204 No Content（ノーコンテント）**「処理は成功したが、本文（返すデータ）は無い」を表す番号。削除の成功時によく使われます。
>
> ⚠️ **member_views.py 全体のパターンを掴もう** 4つのクラスを通して見ると、本アプリのビューには共通の型があります。①セッションで門番（ログイン・権限）→ ②対象データの取得 → ③シリアライザで翻訳・検査 → ④適切なステータスコードで応答。この「型」さえ覚えれば、他の `kosu_views.py` や `team_views.py` も同じ読み方で理解できます。

6. 実コード全解説：serializers.py

> ▼ **このセクションでわかること:** シリアライザが「Pythonのオブジェクト」と「JSON」を双方向に翻訳する係であること、そして本アプリの全シリアライザが驚くほど短く書けている理由を理解します。

`serializers.py` の全文を見ます。

6.1 import 部分 — 1〜3行目

> ▼ **このコードがやること（先に日本語で）:** DRFのシリアライザ部品と、本アプリの全モデルを読み込みます。各シリアライザが「どのモデルを翻訳するか」を指定するために、モデルが必要なのです。

コピー

```
# DRFのシリアライザ部品を読み込む
from rest_framework import serializers
# アプリの各モデルを読み込む（行末の\は行継続）
from ..models import member, Business_Time_graph, kosu_division, def_choice, \
                     # 続き
                     team_member, inquiry_data, administrator_data, AsyncTask, History
```

- **1行目** `from rest_framework import serializers` … DRFの `serializers`（シリアライザ）モジュールを読み込みます。
- **2〜3行目** … 本アプリの全モデル（member, Business_Time_graph …）を `kosu/models.py` から読み込みます。行末の `\`（バックスラッシュ）は「この行はまだ次の行に続く」という継続の印。長い import を見やすく折り返しているだけです。

6.2 MemberSerializer — 7〜10行目

> ▼ **このコードがやること（先に日本語で）:** memberモデルを、JSONに翻訳する係を定義します。たった4行ですが、これだけで「member の全項目をJSONにする・JSONをmemberに戻す・検査する」を全部こなします。

コピー

```
# ModelSerializerを継承した翻訳係
class MemberSerializer(serializers.ModelSerializer):
  # 設定をまとめる入れ子クラス
  class Meta:
    # 翻訳対象はmemberモデル
    model = member
    # 全フィールドを対象にする
    fields = '__all__'
```

▶ ここで初めて出てくるコード

- `__all__`（Django）：`fields = '__all__'` は「モデルの項目を全部使う」という指定です（シリアライザーやフォームで）。

- **7行目** `class MemberSerializer(serializers.ModelSerializer):`
- `ModelSerializer`（モデルシリアライザ）を継承しています。これは「モデルを見て、自動で翻訳ルールを作ってくれる」便利なクラス。普通のシリアライザだと1項目ずつ書く必要がありますが、これなら数行で済みます。

用語: ModelSerializer（モデルシリアライザ）「モデルの設計図を見て、翻訳ルールを自動生成するシリアライザ」。モデルに「従業員番号は整数」「名前は文字列」と書いてあれば、それをそのまま翻訳・検査ルールに使ってくれます。自分でいちいち書かなくて済む省力ツール。

- **8行目** `class Meta:`
- `Meta`（メタ）は「このシリアライザの設定を書く入れ子のクラス」。中に「どのモデルを・どの項目を」翻訳するかを書きます。

用語: Meta クラス（メタクラス）「本体の付帯設定をまとめる小部屋」。シリアライザやモデルの中に `class Meta:` という入れ子を作り、対象モデルや対象項目などの設定を書きます。本体の動作そのものではなく「設定メモ」だと思ってください。

- **9行目** `model = member` … 翻訳対象は `member` モデルだと指定。
- **10行目** `fields = '__all__'` … `'__all__'`（オール）は「**モデルの全項目を対象にする**」という指定。従業員番号・名前・所属・24個の休憩欄…すべてが自動で翻訳対象になります。一部だけにしたい場合は `fields = ['employee_no', 'name']` のようにリストで列挙します。

6.3 残りのシリアライザ — 14〜66行目

> **▼ このコードがやること（先に日本語で）**：他のモデル（工数区分・工数データ・チーム・問い合わせ・管理者設定・タスク・履歴）についても、まったく同じ形で翻訳係を定義しています。中身はモデル名が違うだけです。

コピー

```
# 工数区分定義(kosu_division)の翻訳係
class DefSerializer(serializers.ModelSerializer):
  class Meta:
    model = kosu_division
    fields = '__all__'

# 区分の選択肢(def_choice)の翻訳係
class DefChoiceSerializer(serializers.ModelSerializer):
  class Meta:
    model = def_choice
    fields = '__all__'

# 工数データ(Business_Time_graph)の翻訳係
class KosuSerializer(serializers.ModelSerializer):
  class Meta:
    model = Business_Time_graph
    fields = '__all__'

# チーム(team_member)の翻訳係
class TeamSerializer(serializers.ModelSerializer):
  class Meta:
    model = team_member
    fields = '__all__'

# 問い合わせ(inquiry_data)の翻訳係
class InquirSerializer(serializers.ModelSerializer):
  class Meta:
    model = inquiry_data
    fields = '__all__'

class AdministratorSerializer(serializers.ModelSerializer):# 管理者設定(administrator_data)の翻訳係
  class Meta:
    model = administrator_data
    fields = '__all__'

# 非同期タスク(AsyncTask)の翻訳係
class TaskSerializer(serializers.ModelSerializer):
  class Meta:
    model = AsyncTask
    fields = '__all__'

# 変更履歴(History)の翻訳係
class HistorySerializer(serializers.ModelSerializer):
  class Meta:
    model = History
    fields = '__all__'
```

- **14〜17行目** `DefSerializer` … 工数区分定義 `kosu_division` の翻訳係。
- **21〜24行目** `DefChoiceSerializer` … 工数区分の選択肢 `def_choice` の翻訳係。

- **28〜31行目** `KosuSerializer` … 工数データ本体 `Business_Time_graph` の翻訳係。本アプリの中心データです。
- **35〜38行目** `TeamSerializer` … チーム編成 `team_member` の翻訳係。
- **42〜45行目** `InquirSerializer` … 問い合わせ `inquiry_data` の翻訳係。
- **49〜52行目** `AdministratorSerializer` … 管理者設定 `administrator_data` の翻訳係。
- **56〜59行目** `TaskSerializer` … 非同期タスク `AsyncTask` の翻訳係。
- **63〜66行目** `HistorySerializer` … 変更履歴 `History` の翻訳係。`model = History`、`fields = '__all__'`。

> **なぜ全部こんなに短いの？** `ModelSerializer` + `fields = '__all__'` の組み合わせが強力だからです。モデルさえきちんと作ってあれば、翻訳・検査ルールは全部自動。これがDjango/DRFの「楽さ」の正体です。本アプリは全モデルでこのパターンを徹底しています。
>
> ⚠️ **`'__all__'` の便利さと注意点** 全項目を出すので楽ですが、「外部に見せたくない項目（パスワードなど）」まで出てしまう危険もあります。本アプリの member には機密項目が無いので問題ありませんが、もし秘密の欄を追加したら、`fields` をリスト指定に変えて隠す配慮が必要です。

---

7. 実コード全解説：urls.py（配線盤）

> ▼ **このセクションでわかること:** URLとビューを結びつける「配線盤」の読み方。`path` の3要素、`<int:pk>`、`as_view()`、最後の `static()` の意味を理解します。

`urls.py` は「どのURLが来たら、どのビューに繋ぐか」を一覧にしたファイルです。電話の交換台のようなものです。

7.1 import 部分 — 1〜10行目

> ▼ **このコードがやること（先に日本語で）:** URL定義に必要な `path` 関数と、振り分け先となる各ビューモジュール、そして本番でメディアファイルを配信するための部品を読み込みます。

コピー

```
# URLを定義するpath関数を読み込む
from django.urls import path
# メイン系ビューを読み込む
from .views import main_views
# 工数系ビューを読み込む
from .views import kosu_views
# 人員系ビューを読み込む
from .views import member_views
# チーム系ビューを読み込む
from .views import team_views
# 工数区分定義系ビューを読み込む
from .views import def_views
# 問い合わせ系ビューを読み込む
from .views import inquiry_views
# 非同期処理系ビューを読み込む
from kosu.views import asynchronous_views
# 設定（MEDIA_URL等）を読み込む
from django.conf import settings
# メディア配信用のstaticを読み込む
from django.conf.urls.static import static
```

- **1行目** `from django.urls import path` … URLを1本定義する `path`（パス）関数を読み込みます。
- **2〜8行目** … 振り分け先になる各ビューモジュールを読み込みます。機能ごとにファイルが分かれているので、それぞれ読み込みます。
- **9〜10行目** … 後で（105行目で）メディアファイル配信に使う `settings` と `static` を読み込みます。

7.2 urlpatterns（URLの一覧）— 14行目〜

> **▼ このコードがやること（先に日本語で）：** `urlpatterns` というリストに、URLとビューの対応を1行ずつ並べます。リストの上から順に照合され、最初に一致したものが採用されます。

なぞってコピーすると「…」が別の文字になります。正しくは → コピー

```
# URL対応表の始まり（リスト）
urlpatterns = [
  # /login/ → Loginビュー
  path('login/', main_views.Login.as_view(), name='login'),
  # /logout/ → Logoutビュー
  path('logout/', main_views.Logout.as_view(), name='logout'),
  # …（中略）…
  # /member_list/ → MemberList
  path('member_list/', member_views.MemberList.as_view(), name='member_list'),
  # /member_new/ → MemberNew
  path('member_new/', member_views.MemberNew.as_view(), name='member_new'),
  # /member_update/123/ → MemberUpdate
  path('member_update/<int:pk>/', member_views.MemberUpdate.as_view(), name='member_update'),
  # /member_delete/123/ → MemberDelete
  path('member_delete/<int:pk>/', member_views.MemberDelete.as_view(), name='member_delete'),
  # …（中略：他機能のURLが多数続く）…
]
```

`path()` には3つの要素を渡します。本章の主役、人員系の4行を例に読み解きます。

- **第1要素（URLの形）** `'member_list/'`
- ブラウザがこのパスに来たら、という「URLのパターン」。
- **第2要素（担当ビュー）** `member_views.MemberList.as_view()`
- `MemberList` クラスを「URLに繋げる形」に変換しています。
- `as_view()`（アズ・ビュー）が、その変換を行う決まり文句です（次項で詳説）。
- **第3要素（名前）** `name='member_list'`
- このURLに付ける「あだ名」。プログラムの他の場所から「`member_list` という名前のURL」と呼び出せます。直接URL文字列を書くより、名前で参照するほうが、後でURLを変えても壊れにくくなります。

7.3 as_view() とは ― なぜ必要か

用語: as_view()（アズ・ビュー）「クラスベースのビューを、URLに繋げられる関数の形に変換する」決まりのメソッド。`MemberList` はクラスですが、URLは「関数」を期待しています。そのギャップを埋めるのが `as_view()`。クラスを関数に「変身」させる魔法だと思ってください。

なぜ as_view() が要るの？ Djangoの内部では、URLに対して「呼び出せる関数」を登録する仕組みになっています。`MemberList` はクラスなので、そのままでは登録できません。`as_view()` を付けると「リクエストが来たら、適切な `get` / `post` / `put` / `delete` を呼び分ける関数」が作られ、それが登録されます。

7.4 `<int:pk>` とは — URLから値を受け取る

> **▼ このコードがやること（先に日本語で）：** `member_update/<int:pk>/` の `<int:pk>` は、URLの一部を「変数」として受け取る書き方です。`/member_update/123/` にアクセスすると、`123` が整数として `pk` という名前でビューに渡されます。

- `<int:pk>` の分解：
- `< >` …「ここは固定文字ではなく、変わる部分（変数）だよ」の合図。
- `int` …「ここに来るのは **整数** だよ」という型の指定。`int` 以外に `str`（文字列）、`slug` などもあります。
- `pk` … 受け取った値を入れる **変数名**。ビューの `def put(self, request, pk):` の `pk` にそのまま渡されます。

> ⚠️ **URLの `pk` とビューの引数名は一致させる** URLで `<int:pk>` と書いたら、ビューのメソッドも `def put(self, request, pk):` と、同じ `pk` という名前で受け取る必要があります。名前が食い違うとエラーになります。§5.10で見た `MemberUpdate.put(self, request, pk)` の `pk` が、まさにこのURLから渡される値です。
>
> **▼ こう動けば成功:** ブラウザが `PUT /api/member_update/123/` を送ると… 1. `urls.py` が `member_update/<int:pk>/` と照合し、`pk=123` を取り出す 2. `MemberUpdate.as_view()` が「PUTなので put メソッドを呼ぶ」と判断 3. `MemberUpdate.put(self, request, pk=123)` が実行される という流れで、123番の人員が更新されます。

7.5 非同期処理のURL（同じビューに多数のURL） — 79〜100行目

> **▼ このコードがやること（先に日本語で）：** バックアップ・復元系のURLが大量に並んでいますが、**すべて同じ `asynchronous_views.backup` 関数に繋がっています。** 代わりに `name=`（あだ名）を1本ずつ変えてあり、ビュー側はこの名前を見て「今どの処理か」を判断します（§11で詳説）。

なぞってコピーすると「…」が別の文字になります。正しくは → コピー

```
# 工数バックアップ
path('kosu_backup/', asynchronous_views.backup, name='kosu_backup'),
# 工数削除（同じbackup関数）
path('kosu_delet/', asynchronous_views.backup, name='kosu_delet'),
# 工数復元（同じbackup関数）
path('kosu_load/', asynchronous_views.backup, name='kosu_load'),
# …（中略：def, choice, member, team, inquiry, setting, AsyncTask, History の backup/load が続く）…
# 状態確認
path('check_backup_status', asynchronous_views.check_task_status, name='check_member_backup_status'),
# ダウンロード
path('download_backup', asynchronous_views.download_file, name='download_member_backup'),
```

- **第2要素が関数のまま** … ここでは `asynchronous_views.backup` のように、`.as_view()` を付けていません。これは `backup` が **クラスではなく関数** だからです（§11参照）。関数ビューはそのまま書けます。

- **同じ関数・違う名前** … 18本ほどのURLがすべて `backup` を指しますが、`name=` だけが異なります。ビュー側で `request` から「今のURLの名前」を読み取り、処理を分岐します（§11.2）。

7.6 最後の static() — 105行目

> **▼ このコードがやること（先に日本語で）**：バックアップで作ったファイル（メディアファイル）を、ブラウザからダウンロードできるようにURLを追加します。

コピー

```
# メディア配信用URLを末尾に追加
urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

- `urlpatterns += ...` … 既存のURL一覧に、メディア配信用のURLを **付け足し** ています。
- `static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)` … `MEDIA_URL`（`/media/`）に来たアクセスを、`MEDIA_ROOT`（実ファイルの保存フォルダ）に繋ぎます。これにより、生成したバックアップExcelをダウンロードできます。`MEDIA_URL` と `MEDIA_ROOT` の中身は§12で確認します。

---

8. 実コード全解説：CustomPagination（main_utils.py）

> **▼ このセクションでわかること:** 一覧を「20件ずつ」などに分けるページ送りの仕組みと、本アプリが「1ページの件数を管理者設定から動的に決めている」工夫を理解します。
>
> **用語: ページネーション（pagination：ページネーション）** 大量のデータを「1ページ◯件ずつ」に分けて表示すること。Google検索の結果が10件ずつページ分けされているのと同じ。全件を一度に返すと重いので、必要な分だけ切り出します。

`main_utils.py` の前半（CustomPagination）を読みます。

8.1 import — 1〜6行目

> **▼ このコードがやること（先に日本語で）**：ページネーションの土台クラスや、応答を作る部品、そして「1ページの件数」を読み取るための管理者設定モデルを読み込みます。

コピー

```
# クラスを調べる標準ツール（後半の別関数で使用）
import inspect
# ページ番号方式の土台クラス
from rest_framework.pagination import PageNumberPagination
# 応答を作る部品
from rest_framework.response import Response
# モデルの基底（後半の別関数で使用）
from django.db import models
# アプリのmodelsモジュール全体（別名myapp_models）
from kosu import models as myapp_models
# 管理者設定モデル（1ページ件数の保管先）
from ..models import administrator_data
```

- **2行目** `PageNumberPagination` （ページナンバーページネーション）… 「1ページ目・2ページ目…」とページ番号で送る方式の土台クラス。これを継承します。
- **6行目** `administrator_data` … 管理者設定モデル。ここに「1ページ何件表示するか（menu_row）」が保存されています。

8.2 CustomPagination クラス — 11〜25行目

> **▼ このコードがやること（先に日本語で）**：ページ送りの設定を、固定値ではなく **管理者の設定値から動的に** 決めるクラスです。作られた瞬間に管理者設定の最新レコードを見て「1ページ何件か」を取り込み、応答には「全件数・1ページ件数・データ本体」を含めます。

コピー

```
# ページネーションクラス
# PageNumberPaginationを継承
class CustomPagination(PageNumberPagination):
  # デフォルトの設定値                          # 既定は1ページ20件
  page_size = 20

  # このクラスが作られた瞬間に動く初期化メソッド
  def __init__(self):
    # administrator_data から動的にページサイズを設定
    # 管理者設定の最新レコードを取得
    last_record = administrator_data.objects.order_by("id").last()
    # 設定が存在すれば
    if last_record is not None:
      # その値で1ページ件数を上書き
      self.page_size = last_record.menu_row

  # 応答の中身を組み立てるメソッド
  def get_paginated_response(self, data):
    return Response({
      # 合計件数            # 全部で何件あるか
      'count': self.page.paginator.count,
      # ページサイズをレスポンスに含める  # 1ページ何件か
      'page_size': self.page_size,
      # 現在のページのデータ                  # 今のページのデータ本体
      'results': data,
    })
```

- **11行目** `class CustomPagination(PageNumberPagination):` … 標準のページネーションを継承し、独自版を作ります。
- **12行目** `page_size = 20` … 既定値（デフォルト）は20件。後で上書きされなければこの値が使われます。
- **14行目** `def __init__(self):`
- `__init__`（イニット）は「インスタンスが作られた瞬間に自動で呼ばれる初期化メソッド」。`paginator = CustomPagination()`（§5.7）の `()` の瞬間に、ここが走ります。

> **用語: `__init__`（イニット＝初期化メソッド）／コンストラクタ** クラスから実体（インスタンス）が作られる瞬間に1回だけ自動実行される、準備のためのメソッド。前後のアンダースコア2個（ダンダー）が目印。家を建てた瞬間に「電気と水道を通す」初期工事のイメージ。

- **16行目** `last_record = administrator_data.objects.order_by("id").last()`
- 管理者設定を `id` 順に並べ、`.last()` で **一番最後（最新）の1件** を取得します。これが現在有効な設定です。

- **17〜18行目** … 設定が存在すれば（`None` でなければ）、その `menu_row`（1ページ表示件数）で `self.page_size` を上書きします。

> **なぜ件数を設定から取るの？**「1ページに何件表示するか」を、コードを書き換えずに **管理画面から変えられる** ようにするためです。管理者が「1ページ30件にしたい」と思ったら、設定値を変えるだけ。次に作られる `CustomPagination` から自動で反映されます。

- **20行目** `def get_paginated_response(self, data):`
- §5.7の最後で呼ばれていたメソッド。標準では `next` / `previous`（次/前ページのURL）が含まれますが、本アプリは独自に上書きして、`count`・`page_size`・`results` の3つだけのシンプルな形にしています。
- **22行目** `'count': self.page.paginator.count` … 絞り込み後の **全件数**。Reactがページ番号の総数を計算するのに使います。
- **23行目** `'page_size': self.page_size` … 1ページの件数も応答に含め、React側が把握できるようにしています。
- **24行目** `'results': data` … 今のページのデータ本体（シリアライズ済みのJSON）。

> **▼ §5.7の応答例（再掲）と対応:**
>
> コピー
>
> ```
> { "count": 53, "page_size": 20, "results": [ ... ] }
> ```
>
> この `count`・`page_size`・`results` は、まさにこの `get_paginated_response` が組み立てたものです。
>
> **補足:** `main_utils.py` には他に `validate_employee_no_logic`（従業員番号の検証）や `get_all_model_names_in_myapp`（全モデル名取得）という関数もありますが、これらはバックアップ系やチーム系の別機能で使われるユーティリティです。本章の主題（人員ビューとページネーション）の本筋ではないので、ここでは深入りしません。

---

9. 実コード全解説：signals.py（自動の変更履歴）

> **▼ このセクションでわかること:** 本アプリの「誰が・いつ・どのデータを・どう変えたか」を自動記録する仕組み。ビューに履歴処理を一切書かなくても、保存・削除のたびに `History` テーブルへ記録が残る「魔法」の正体を理解します。
>
> **用語: シグナル（signal：シグナル）**「ある出来事が起きたとき、自動で別の処理を呼び出す」Djangoの仕組み。たとえば「データが保存された直後（post_save）」という出来事に、「履歴を残す処理」を結びつけておくと、保存のたびに自動で履歴が残ります。火災報知器が「煙を感知したら自動でベルを鳴らす」のと同じ。プログラマが毎回「履歴を残せ」と命令しなくてよくなります。

9.1 import とスレッドローカル — 1〜11行目

> **▼ このコードがやること（先に日本語で）**：シグナルに必要な部品（pre_save等）と、履歴を残す対象モデル、現在のリクエストを取り出す関数を読み込みます。さらに「更新前の値を一時的に覚えておく入れ物」を用意します。

コピー

```
# スレッドごとの保管庫を作る部品
from threading import local
# 3つのシグナル（保存前・保存後・削除後）
from django.db.models.signals import pre_save, post_save, post_delete
# シグナルを受け取る目印デコレータ
from django.dispatch import receiver
# 対象モデル群とHistory
from .models import History, member, Business_Time_graph, team_member, kosu_division, def_choice, administrator_data, inquiry_data
# 現在のリクエストを取り出す関数
from .middleware.clear_session_middleware import get_current_request
# モデルの型判定用
from django.db import models

# スレッドローカル変数初期化
# 更新前の値を一時保管する入れ物
_thread_locals = local()
```

- **2行目** … 3つのシグナルを読み込みます。
- `pre_save`（プリ・セーブ）＝ 保存の **直前** に発火。
- `post_save`（ポスト・セーブ）＝ 保存の **直後** に発火。
- `post_delete`（ポスト・デリート）＝ 削除の **直後** に発火。
- **3行目** `receiver`（レシーバー）＝「この関数はこのシグナルの受け手だよ」と印を付けるデコレータ（後述）。
- **5行目** `get_current_request` …「今処理中のリクエスト」を取り出す関数（§10で詳説）。これで「誰が操作したか」を知ります。
- **11行目** `_thread_locals = local()` … **スレッドローカル**（後述）という、リクエストごとに独立した一時保管庫を作ります。更新前の値をここに一時的にしまいます。

> **用語: スレッドローカル（thread-local：スレッドローカル）**「処理の流れ（スレッド）ごとに別々の値を持てる保管庫」。サーバーは複数のリクエストを同時にさばくので、Aさんの更新前データとBさんの更新前データが混ざると大事故です。スレッドローカルなら、各リクエスト専用の引き出しに分けて保管できるので混ざりません。会社の「各人の専用ロッカー」のイメージ。

9.2 更新前の値をキャッシュする2関数 — 14〜27行目

> **▼ このコードがやること（先に日本語で）**：「保存する直前に、変更前の値をデータベースから取って一時保管しておく」関数と、「その保管した値を後で取り出す」関数のペアです。更新後に「どこがどう変わったか（差分）」を計算するために、変わる前の姿を覚えておく必要があるのです。

コピー

```
# 更新前の値をスレッドローカルキャッシュに保存
# 更新前の値を保管する関数
def set_instance_cache(instance):
  # そのデータが何のモデルか調べる
  model = type(instance)
  try:
    # 更新前の値取得→スレッドローカルキャッシュに保存
    # DBから更新前の値を取り、ロッカーにしまう
    _thread_locals.instance_cache = model.objects.get(pk=instance.pk)
  # 新規でまだDBに無い場合
  except model.DoesNotExist:
    # レコードなしの場合はNone取得
    # Noneをしまう
    _thread_locals.instance_cache = None

# スレッドローカルキャッシュから更新前の値取得
# 保管した更新前の値を取り出す関数
def get_instance_cache():
  # ロッカーから取り出す（無ければNone）
  return getattr(_thread_locals, 'instance_cache', None)
```

- **14行目** `def set_instance_cache(instance):`
- `instance`（インスタンス）は「これから保存される、1件のデータ」。
- **15行目** `model = type(instance)` … その `instance` が何のモデルか（member なのか kosu なのか）を調べます。
- **18行目** `_thread_locals.instance_cache = model.objects.get(pk=instance.pk)`
- **まだ保存する前** に、データベースに残っている **古い値** を取得し、ロッカー（スレッドローカル）にしまいます。これが「変更前の姿」のスナップショットです。
- **19〜21行目** … 新規作成で、まだDBに古い値が無い場合は `None` をしまいます。
- **26〜27行目** `get_instance_cache()` … しまっておいた更新前の値を取り出します。`getattr(..., None)` で「無ければ None」を返す安全な取り出し方です。

9.3 差分を計算する get_changes — 32〜106行目

ここが履歴機能の頭脳です。「更新前」と「更新後」を全項目で比べ、変わったところだけを記録用にまとめます。少し長いので段階的に読みます。

9.3.1 キャッシュの取得と安全確認 — 32〜40行目

> **▼ このコードがやること（先に日本語で）**：差分を入れる空の入れ物を用意し、§9.2でしまった「更新前の値」を取り出します。さらに、取り出した古い値が「本当に今のデータと同じ種類・同じ1件のものか」を確認し、もし食い違っていたら誤比較を避けるため捨てます（None扱い）。

コピー

```
# 値の差分取得
# 差分を計算する関数（created=新規かどうか）
def get_changes(instance, created):
  # 変更点を入れる空の辞書
  changes = {}

  # 1. キャッシュを取得
  # 更新前の値を取り出す
  old_instance = get_instance_cache()

  # 【修正】キャッシュが存在しても、現在のインスタンスと型やPKが違う場合は、誤った比較を避けるためNoneにする
  # 種類かPKが食い違うなら
  if old_instance and (not isinstance(old_instance, type(instance)) or old_instance.pk != instance.pk):
    # 信用せず捨てる
    old_instance = None
```

- **32行目** `def get_changes(instance, created):`
- `created`（クリエイテッド）は「これは新規作成か？」を表す True/False。新規なら全項目が「新しい値」、更新なら「変わった項目だけ」を記録する、という分岐に使います。
- **33行目** `changes = {}` … 変更内容をためる空の辞書。
- **36行目** `old_instance = get_instance_cache()` … §9.2でしまった更新前の値を取り出します。
- **39行目** … `isinstance(old_instance, type(instance))`（種類が同じか）と `old_instance.pk != instance.pk`（同じ1件か）を確認。どちらか食い違えば、別物を比べてしまう危険があるので `old_instance = None` にして捨てます。これは複数リクエストが混線したときの保険（コメントの「【修正】」が示すように、後から補強された安全策）です。

9.3.2 全フィールドを1つずつ比較 — 43〜67行目

> **▼ このコードがやること（先に日本語で）**：モデルの全項目を1つずつ回り、「新しい値」と「古い値」を取り出します。新規なら古い値はNone・全部変更扱い。更新なら、キャッシュがあればそれと比較、無ければ念のためDBから直接古い値を取り直して比較します。

コピー

```
# モデルの全フィールド処理
# モデルの全項目を1つずつ回す
for field in instance._meta.fields:
  # 項目名を取り出す
  field_name = field.name

  # 値取得
  # 新しい値（更新後）を取り出す
  new_value = getattr(instance, field_name)

  # 新規作成時、全て変更として処理
  # 新規作成なら
  if created:
    # 古い値は無い
    old_value = None
    # 全項目を変更扱いにする
    is_changed = True
  # 更新時、差分取得
  # 更新なら
  else:
    # キャッシュ(更新前)があれば
    if old_instance:
      # そこから古い値を取り出す
      old_value = getattr(old_instance, field_name)
      # 新旧が違えば変更ありと判定
      is_changed = (old_value != new_value)
    # キャッシュが無ければ（保険）
    else:
      # 【修正】pre_saveでのキャッシュ漏れやスレッドの混同対策としてDBから直接取得を試みる
      try:
        # DBから古い値を取り直す
        old_db_instance = type(instance).objects.get(pk=instance.pk)
        # その項目の古い値
        old_value = getattr(old_db_instance, field_name)
        # 比較
        is_changed = (old_value != new_value)
        # 以降の処理（Relation等）で使うためキャッシュを一時的に更新
        # 取り直した値を以降で使う
        old_instance = old_db_instance
      # それでも無ければ
      except type(instance).DoesNotExist:
        # この項目は飛ばす
        continue
```

- **43行目** `for field in instance._meta.fields:`
- `instance._meta.fields` は「そのモデルの全項目（フィールド）の一覧」。`for` で1項目ずつ順番に処理します。
- **44行目** `field_name = field.name` … 今見ている項目の名前（例: `name`, `shop`）。

- **47行目** `new_value = getattr(instance, field_name)` … `getattr` で、その項目の **新しい値** を動的に取り出します。

> **用語: getattr（ゲットアター）** `getattr(オブジェクト, '項目名')` で「オブジェクトの、その名前の項目の値」を取り出す関数。`instance.name` と書く代わりに、項目名が変数のときに使えます。全項目をループで回すこの場面で大活躍します。

- **50〜52行目** … 新規作成（`created` が True）なら、古い値は `None`、全項目を「変更あり（is_changed=True）」として扱います。
- **55〜57行目** … 更新で、キャッシュがあれば、そこから古い値を取り、新旧を `!=`（違うか）で比較。違えば変更ありと判定します。
- **59〜67行目** … キャッシュが無い保険ルート。DBから古い値を取り直して比較し、以降の処理でも使えるよう `old_instance` に入れ直します。それでも見つからなければ `continue`（この項目は飛ばす）。

9.3.3 値をJSON保存できる形に変換 — 69〜97行目

> **▼ このコードがやること（先に日本語で）**：変更があった項目について、その値を「JSONとして安全に保存できる形」に整えます。他テーブルへのつながり（リレーション）や日付などはそのままだとJSONにできないので、ID・文字列・isoformat文字列などに変換します。

コピー

```
    # 変更or新規作成
    # 変更があった項目だけ処理
    if is_changed:

      # JSON化できないリレーションフィールド処理
      # 他テーブルへの参照（多対一）なら
      if field.is_relation and field.many_to_one:
        # 関連ID取得
        # 古い側の関連ID
        old_json_safe_value = getattr(old_instance, field.attname) if old_instance else None
        # 新しい側の関連ID
        new_json_safe_value = getattr(instance, field.attname)

      # その他オブジェクト処理
      # 基本型でない（複雑な）値なら
      elif not isinstance(new_value, (str, int, float, bool, type(None))):
        # インスタンスが直接フィールドに格納されている場合、IDと文字列表現記録
        # モデルそのものなら
        if isinstance(new_value, models.Model):
          # 古い側をid+文字列で
          old_json_safe_value = {'id': old_value.pk, 'str': str(old_value)} if old_value else None
          # 新しい側をid+文字列で
          new_json_safe_value = {'id': new_value.pk, 'str': str(new_value)}
        # Datetimeの場合の処理
        # 日付/日時なら
        elif isinstance(new_value, (models.DateField, models.DateTimeField)):
          # 古い側を標準文字列で
          old_json_safe_value = old_value.isoformat() if old_value else None
          # 新しい側を標準文字列で
          new_json_safe_value = new_value.isoformat()
        # その他は文字列化
        # それ以外の複雑な値は
        else:
          # 文字列に
          old_json_safe_value = str(old_value) if old_value else None
          # 文字列に
          new_json_safe_value = str(new_value)

      # JSON化可能な基本データ型の処理
      # 文字列・数値などの基本型は
      else:
        # そのまま
        old_json_safe_value = old_value
        # そのまま
        new_json_safe_value = new_value
```

- **70行目** `if is_changed:` … 変更があった項目だけが、この中の処理に入ります。変わっていない項目は記録しません（無駄を省く）。
- **73〜76行目** `if field.is_relation and field.many_to_one:`
- その項目が「他テーブルへのつながり（多対一のリレーション）」なら、関連先のオブジェクトそのものは重くてJSONにできないので、**その関連ID（`field.attname`、たとえば `member_id`）** だけを記録します。

用語: リレーション（relation：リレーション）／多対一（many-to-one） テーブル同士のつながり。「多対一」は「多くの工数記録が、1人の人員に紐づく」ような関係。工数記録（多）→ 人員（一）。詳しくは第8章で扱いますが、ここでは「他テーブルへの参照は、つながり先のIDだけ控える」と理解すれば十分です。

- **79〜91行目** `elif not isinstance(new_value, (str, int, float, bool, type(None))):`
- 値が「文字列・整数・小数・真偽・None」のような単純な型 **でない**（複雑な）場合の処理。
- **81〜83行目** … モデルそのものなら、`{'id': ..., 'str': ...}`（IDと文字列表現）の形で記録。
- **85〜87行目** … 日付・日時なら、`.isoformat()` で「2026-06-02」のような標準的な文字列に変換して記録。
- **89〜91行目** … それ以外の複雑な値は、とりあえず `str(...)` で文字列にして記録。
- **94〜96行目** `else:` … 文字列や数値などの基本型は、変換不要なのでそのまま使います。

なぜこんなに変換が必要なの？ 履歴は `changes` という欄に **JSON形式** で保存されます。JSONは「文字列・数値・真偽・リスト・辞書」しか入れられません。日付オブジェクトや、他テーブルへのつながりは、そのままだとJSONにできずエラーになるので、文字列やIDに「JSONが食べられる形」へ変換しているのです。

9.3.4 changes 辞書に記録して返す — 98〜106行目

▼ このコードがやること（先に日本語で）：変換した値を、変更点をためる辞書に書き込みます。新規作成のときは「新しい値」だけ、更新のときは「古い値と新しい値のペア」を記録します。最後にこの辞書を返します。

コピー

```
    # 6. changes辞書に記録
    # 新規作成なら
    if created:
      # 新規作成時は新しい値のみ記録
      # 新しい値だけ記録
      changes[field_name] = new_json_safe_value
    # 更新なら
    else:
      # 更新時は古い値と新しい値を記録
      # 古い値と新しい値をペアで記録
      changes[field_name] = {'old': old_json_safe_value, 'new': new_json_safe_value}

  # 完成した差分を返す
  return changes
```

- **99〜101行目** … 新規作成では、項目名 → 新しい値、という形で記録（古い値は無いので）。
- **102〜104行目** … 更新では、項目名 → `{'old': 古い値, 'new': 新しい値}` という形で、変更前後をペアで記録します。後から「名前が『山田』→『田中』に変わった」と一目で分かります。
- **106行目** `return changes` … 完成した差分辞書を返します。

▼ **こんな差分が記録される（※説明用の簡易例）**：「山田さんの所属を W1 → A1 に変更」した場合、changes はこうなります。

コピー

```
{ "shop": {"old": "W1", "new": "A1"} }
```

変わった `shop` だけが、変更前後つきで記録されます。これが `History` テーブルに残り、管理画面で監査できます。

9.4 シグナルの登録（receiver デコレータ） — 111〜159行目

▼ **このコードがやること（先に日本語で）**：ここまで作った部品を、実際に「保存の前後・削除の後」というタイミングに結びつけます。member モデルを例に、保存前のキャッシュ・保存後の履歴記録・削除後の履歴記録の3つを登録します。

コピー

```
# 保存前に更新前の値をキャッシュ
# 「memberの保存直前」にこの関数を結びつける
@receiver(pre_save, sender=member)
# 保存前に呼ばれる
def cache_old_member_instance(sender, instance, **kwargs):
  # 更新前の値をロッカーにしまう
  set_instance_cache(instance)
```

- **111行目** `@receiver(pre_save, sender=member)`
- `@`（アットマーク）で始まるのが **デコレータ**（後述）。「すぐ下の関数を、`member` の `pre_save`（保存直前）に結びつけよ」という意味。
- これにより、member が保存される **直前** に、自動で `cache_old_member_instance` が呼ばれます。

用語: デコレータ（decorator：デコレータ） 関数の上に `@○○` と書いて、その関数に追加の役割を「貼り付ける」仕組み。ここでは「この関数をシグナルの受け手にする」という役割を貼っています。プレゼントにリボン（飾り）を付けるように、関数に機能を装飾するイメージ。

- **112行目** `def cache_old_member_instance(sender, instance, **kwargs):`
- シグナルから呼ばれる関数。`sender`（送り手＝どのモデルか）、`instance`（対象データ）、`**kwargs`（その他の付帯情報をまとめて受ける）を受け取ります。
- **113行目** `set_instance_cache(instance)` … §9.2の関数を呼び、更新前の値をしまいます。これが「保存直前」に走るので、上書きされる前の姿を確保できます。

> **用語: `**kwargs` （クワーグズ）** 「キーワード引数を何個でもまとめて受け取る」書き方。シグナルは色々な付帯情報を渡してきますが、使わないものは `**kwargs` でまとめて受け流せます。「その他もろもろ」の受け皿。

なぞってコピーすると全角の空白が別の文字になります。正しくは → コピー

```
# 履歴を記録　新規作成、更新（member）
# 「memberの保存直後」に結びつける
@receiver(post_save, sender=member)
# 保存後に呼ばれる（createdで新規か判定）
def log_create_update_member_history(sender, instance, created, **kwargs):
  # 今のリクエストを取得（誰が操作したか）
  request = get_current_request()
  # 操作者の従業員番号を取得
  session_data = request.session.get('login_No') if request else None

  # 差分計算
  # §9.3で変更点を計算
  changes = get_changes(instance, created)

  # 操作内容判定
  # 新規ならCREATE、更新ならUPDATE
  operation = 'CREATE' if created else 'UPDATE'

  # 履歴記録
  # Historyテーブルに1件作る
  History.objects.create(
    # 操作種別
    operation=operation,
    # 対象テーブル名
    table_name='member',
    # 対象レコードのID
    record_id=instance.id,
    # 操作者
    login_No=session_data,
    # 変更内容
    changes=changes,
  )
```

- **124行目** `@receiver(post_save, sender=member)` … member の **保存直後** にこの関数を結びつけます。
- **125行目** `def log_create_update_member_history(sender, instance, created, **kwargs):`
- `created` を受け取っているのがポイント。新規作成なら True、更新なら False。これで操作種別を判定します。
- **126行目** `request = get_current_request()` … §10の関数で「今処理中のリクエスト」を取得。ここから操作者を知ります。
- **127行目** `session_data = request.session.get('login_No') if request else None`

- リクエストがあればセッションから操作者の従業員番号を取り、無ければ None。`A if 条件 else B` は「条件ならA、でなければB」という1行の書き方（三項演算子）です。
- **130行目** `changes = get_changes(instance, created)` … §9.3で差分を計算。
- **133行目** `operation = 'CREATE' if created else 'UPDATE'` … 新規なら 'CREATE'、更新なら 'UPDATE' の文字列を作ります。
- **136〜142行目** `History.objects.create(...)`
- `History` テーブルに新しい履歴を1件作ります。操作種別・テーブル名・対象ID・操作者・変更内容をまとめて記録。**この1回の create で履歴が永久に残ります。**

なぞってコピーすると全角の空白が別の文字になります。正しくは → コピー

```
# 履歴を記録　削除（member）
# 「memberの削除直後」に結びつける
@receiver(post_delete, sender=member)
# 削除後に呼ばれる
def log_delete_member_history(sender, instance, **kwargs):
  # 今のリクエスト取得
  request = get_current_request()
  # 操作者取得
  session_data = request.session.get('login_No') if request else None

  # 履歴記録
  # 履歴を1件作る
  History.objects.create(
    # 操作はDELETE固定
    operation='DELETE',
    # 対象テーブル
    table_name='member',
    # 削除されたレコードのID
    record_id=instance.id,
    # 操作者
    login_No=session_data,
    # 削除なので変更内容はなし
    changes=None,
  )
```

- **147行目** `@receiver(post_delete, sender=member)` … member の **削除直後** に結びつけます。
- **153〜159行目** … 削除の履歴を記録。操作は `'DELETE'` 固定、`changes=None`（差分は無いので）。削除前のID（`instance.id`）はまだ参照できるので、「誰が何番を消したか」が残ります。

9.5 他のモデルへの同じ登録 — 163〜399行目

> **▼ このコードがやること（先に日本語で）**：member とまったく同じパターンの「保存後・削除後の履歴記録」を、他の主要モデルにも繰り返し登録しています。コードの形はモデル名・テーブル名が違うだけで、構造は同一です。

`signals.py` の残り（117〜399行目）は、次のモデルそれぞれに対して、§9.4とそっくりの `post_save`（新規・更新）と `post_delete`（削除）のペアを登録しています。

対象モデル	post_save（新規・更新）	post_delete（削除）	行
`Business_Time_graph`（工数データ）	あり（差分記録）	あり	117〜199
`team_member`（チーム）	あり	あり	203〜239
`kosu_division`（工数区分定義）	あり	あり	243〜279
`def_choice`（区分の選択肢）	あり	あり	283〜319
`administrator_data`（管理者設定）	あり	あり	323〜359
`inquiry_data`（問い合わせ）	あり	あり	363〜399

> ⚠️ なお、`pre_save`（保存前キャッシュ）が登録されているのは `member`（111行目）と `Business_Time_graph`（117行目）の2モデルだけです。他のモデルでは、§9.3.2で見た「キャッシュが無ければDBから取り直す」保険ルートが働いて差分を計算します。
>
> **なぜ同じコードを何度も書くの？** モデルごとに `@receiver(..., sender=○○)` で **対象を変えて登録する必要がある** からです。共通化する書き方もありますが、本アプリは「読んで分かりやすさ」を優先し、各モデルに明示的に書いています。後任者が「このモデルの履歴はここ」とすぐ見つけられる利点があります。
>
> **▼ 全体像をつかもう（履歴のしくみ）**：1. ビュー（member_views.py）で `serializer.save()` や `.delete()` を呼ぶ 2. その瞬間、Djangoが自動で `pre_save` → `post_save`（または `post_delete`）シグナルを発火 3. signals.py に登録された関数が呼ばれ、`get_changes` で差分を計算 4. `History.objects.create(...)` で履歴を永久保存
>
> **ビューには履歴のコードが1行も無い** のに、全データ操作が自動で監査記録される——これがシグナルの威力です。

---

10. 実コード全解説：middleware と apps.py

10.1 ミドルウェアとは

> **用語: ミドルウェア（middleware：ミドルウェア）** リクエストとビューの「あいだ（middle）」に挟まる中間処理。すべてのリクエストが、ビューに届く前と、応答が返る前に、必ず通過する関所の列。たとえば「セキュリティチェック」「セッション復元」「CORSチェック」などが順番に並びます。空港の「保安検査→出国審査→搭乗」のような一連の関所だと思ってください。

§1の旅で見た「②MIDDLEWARE を順に通過」の部分です。本アプリ独自のミドルウェアが `clear_session_middleware.py` です。

10.2 CurrentRequestMiddleware — clear_session_middleware.py 全文

> **▼ このコードがやること（先に日本語で）**：リクエストが来るたびに、その「今のリクエスト」をスレッド専用の保管庫にしまい、処理が終わったら片付けます。これにより、§9のシグナルのような「リクエストを直接受け取れない場所」でも「今誰が操作しているか」を知ることができます。

コピー

```
# スレッド機能を読み込む
import threading

# スレッドローカルオブジェクト作成
# リクエストをしまうスレッド専用ロッカー
_request_local = threading.local()
# 各リクエスト処理中、現在のHTTPリクエストオブジェクトをスレッドローカルストレージに保存
# 独自ミドルウェアのクラス
class CurrentRequestMiddleware:
  # ミドルウェア初期化
  # サーバー起動時に1回呼ばれる
  def __init__(self, get_response):
    # 次のミドルウェアかget_respons取得
    # 「次の関所」を覚えておく
    self.get_response = get_response

  # リクエスト処理
  # リクエストごとに呼ばれる
  def __call__(self, request):
    # 現在のリクエストオブジェクトをスレッドローカルに保存
    # 今のリクエストをロッカーにしまう
    _request_local.request = request
    # 次のミドルウェアかget_respons取得
    # 次の関所（最終的にビュー）を呼び、応答を得る
    response = self.get_response(request)

    # スレッドローカル内のリクエストオブジェクト削除(メモリリーク、クロススレッドデータ汚染防止)
    # 後始末：ロッカーを空にする
    _request_local.request = None
    # 応答を返す
    return response

# 現在のスレッドのHTTPリクエストオブジェクト取得
# しまったリクエストを取り出す関数
def get_current_request():
  # スレッドローカルストレージから'request'属性値取得
  # 無ければNoneを返す
  return getattr(_request_local, 'request', None)
```

▶ ここで初めて出てくるコード

- `.local()`（Python）：`threading.local()` は、スレッド（同時に動く処理）ごとに別々の中身を持つ入れ物を作ります。同時に来たリクエスト同士で、値が混ざりません。
- `.get_response()`（Django）：ミドルウェア（リクエストの前後に必ず通る処理）で、次の係（最後はビュー）にリクエストを渡して返事を受け取る関数です。
- `__call__`（Python）：`def __call__(self, request):` を書いたクラスの物は、関数のように `物(request)` と呼べます。Django のミドルウェアは、リクエストのたびにこれが呼ばれます。

- **6行目** `_request_local = threading.local()` … リクエストをしまうスレッド専用ロッカー（§9.1のスレッドローカルと同じ考え方）。
- **10行目** `def __init__(self, get_response):`
- ミドルウェアは「`__init__` で初期化、`__call__` で各リクエスト処理」という決まった形です。
- `get_response`（ゲット・レスポンス）は「次の関所（最終的にはビュー）」。これを覚えておきます。
- **12行目** `self.get_response = get_response` … 次の関所を保存。
- **16行目** `def __call__(self, request):`
- `__call__`（コール）は「このオブジェクトが関数のように呼ばれたとき」に動くメソッド。リクエストが来るたびに呼ばれます。
- **18行目** `_request_local.request = request` … 今のリクエストをロッカーにしまいます。これで、§9のシグナルから `get_current_request()` で取り出せるようになります。
- **20行目** `response = self.get_response(request)` … 次の関所（やがてビュー）を呼び、応答を得ます。この行の中で、ビューの処理（§5の全部）が走り、シグナルも発火します。
- **23行目** `_request_local.request = None` … 処理が終わったら、**ロッカーを空にして後始末** します。これを忘れると、次のリクエストに前のデータが残る「混線」や、メモリの無駄遣い（メモリリーク）が起きます。
- **24行目** `return response` … 応答を返します。
- **29〜31行目** `def get_current_request():`
- ロッカーから「今のリクエスト」を取り出す関数。§9のシグナルがこれを呼んで「誰が操作したか」を知ります。無ければ `None`。

なぜこんな回りくどいことを？ シグナル（§9）は「データが保存された」という出来事だけを受け取り、**リクエストは直接受け取れません**。でも「誰が操作したか」はリクエストのセッションにしかありません。そこで、ミドルウェアでリクエストをロッカーにしまっておき、シグナルからそっと取り出す、という橋渡しをしているのです。

⚠️ **ファイル名と中身のズレ** ファイル名は `clear_session_middleware.py`（セッションを消す、という名前）ですが、中身は「リクエストをスレッドに保管する CurrentRequestMiddleware」です。名前と中身が一致していませんが、過去の経緯によるものです。動作には影響しないので、混乱しないよう注意してください。

10.3 settings.py での登録 — MIDDLEWARE の最後

このミドルウェアは、`settings.py` の `MIDDLEWARE` リストの **最後** に登録されています（§12で全体を見ます）。

コピー

```
    # 独自ミドルウェアを最後に登録
    'kosu.middleware.clear_session_middleware.CurrentRequestMiddleware',
```

リストの順番がそのまま「関所を通る順番」です。最後にあるので、セッションが復元された後にリクエストを保管できます。

10.4 apps.py の ready — シグナルを有効化する1行

> **▼ このコードがやること（先に日本語で）**：アプリが起動して準備が整った瞬間に、`signals.py` を読み込みます。これをしないと、§9で書いたシグナル登録（@receiver）が実行されず、履歴が一切残りません。

コピー

```
# アプリ設定の土台クラスを読み込む
from django.apps import AppConfig

# kosuアプリの設定クラス
class KosuConfig(AppConfig):
    # 主キーの既定型（大きな整数の自動採番）
    default_auto_field = 'django.db.models.BigAutoField'
    # アプリ名は kosu
    name = 'kosu'

    # アプリ起動準備完了時に1回呼ばれる
    def ready(self):
        # signals.pyを読み込んでシグナルを登録する
        import kosu.signals
```

> **▶ ここで初めて出てくるコード**
>
> - **`AppConfig`**（Django）：アプリの設定のクラスの土台。`apps.py` で `class KosuConfig(AppConfig):` と書き、`def ready(self):` でシグナルを読み込むなど、起動の時の準備をします。

- **4行目** `class KosuConfig(AppConfig):` … kosu アプリ全体の設定クラス。
- **5行目** `default_auto_field = 'django.db.models.BigAutoField'` … 各テーブルの主キー（背番号）に、大きな整数を自動で振る方式を既定にします。
- **6行目** `name = 'kosu'` … このアプリの名前。
- **8行目** `def ready(self):`

- `ready`（レディ＝準備完了）は「アプリの準備が整った瞬間に1回だけ呼ばれる」メソッド。
- **9行目** `import kosu.signals`
- ここで `signals.py` を読み込みます。**この import が走った瞬間に、§9.4の `@receiver` 群が実行され、シグナルが登録されます。**

> ⚠️ **この1行が無いと履歴が記録されない** `@receiver` デコレータは「そのファイルが読み込まれた瞬間」に登録処理を行います。`signals.py` はどこからも普通には import されないので、ここで意図的に読み込まないと、シグナルが一切有効になりません。「履歴が残らない！」というトラブルの多くは、この `ready` での import 漏れが原因です。

---

11. 実コード全解説：asynchronous_views.py（非同期処理）

> ▼ **このセクションでわかること:** 「時間のかかる処理（数万件のバックアップなど）を、ユーザーを待たせずに裏で走らせる」非同期処理のしくみ。タスクIDを発行し、別スレッドで実行し、ブラウザが後から状態を問い合わせる流れを理解します。
>
> **用語: 非同期処理（asynchronous：エイシンクロナス）とは？** 「時間のかかる処理を裏で走らせ、終わるのを待たずに先に進む」やり方。たとえばレストランで「料理に30分かかる注文」をしたとき、客を厨房の前に立たせ続ける（同期）のではなく、番号札を渡して席で待ってもらい、できたら呼ぶ（非同期）方式。本アプリのバックアップ/復元はこの方式です。

11.1 import — 1〜17行目

> ▼ **このコードがやること（先に日本語で）**：非同期処理に必要な標準ツール（スレッド・一意IDの生成・一時ファイル等）、実際の処理本体（tasks.py の各関数）、タスク状態を保存するモデル、DRFの部品を読み込みます。

コピー

```
# ファイルパス操作
import os
# 別スレッドで処理を走らせる
import threading
# 一意なタスクIDを生成
import uuid
# 日付処理
import datetime
# 待機(sleep)用
import time
# 一時ファイル作成
import tempfile
# 実処理本体（工数系）
from ..tasks import generate_kosu_backup, delete_kosu_data, load_kosu_file, \
                    # 人員・チーム系
                    generate_member_backup, load_member_file, generate_team_backup, load_team_file, \
                    # 区分系
                    generate_def_backup, load_def_file, generate_choice_backup, load_choice_file, \
                    # 問い合わせ・設定系
                    generate_inquiry_backup, load_inquiry_file, generate_setting_backup, load_setting_file, \
                    # タスク・履歴系
                    generate_AsyncTask_backup, delete_AsyncTask_data, generate_History_backup ,delete_History_data
# タスク状態を保存するモデル
from ..models import AsyncTask
# ステータスコード
from rest_framework import status
# 関数ビュー用デコレータ
from rest_framework.decorators import api_view, parser_classes
# 受信形式の解析器
from rest_framework.parsers import MultiPartParser, JSONParser, FormParser
# URLから名前を逆引きする道具
from django.urls import resolve
# JSON応答・ファイル応答
from django.http import JsonResponse, FileResponse
```

▶ ここで初めて出てくるコード

- **`FileResponse`**（Django）: Django の `FileResponse(開いたファイル)` は、ファイルを、少しずつ送る答えです。添付ファイルのダウンロードに使います。

- **3行目** `import uuid` … `uuid`（ユーユーアイディー）は「世界で重複しない一意なID」を作る道具。各タスクに固有の番号札を発行するのに使います。
- **6行目** `import tempfile` … アップロードされたファイルを一時的に保存する「一時ファイル」を作る道具。
- **7〜11行目** … `tasks.py` から、実際のバックアップ/復元/削除を行う関数群を読み込みます。これらが「重い処理の本体」です。
- **12行目** `from ..models import AsyncTask` … タスクの状態（pending/success/error）を保存するモデル。番号札の控えを保管する台帳です。
- **16行目** `from django.urls import resolve` … URLパスから、そのURLの `name`（あだ名）を逆引きする道具。§7.5で「同じ関数・違う名前」だったのを思い出してください。

11.2 backup 関数（非同期処理の入口）— 21〜35行目

> **▼ このコードがやること（先に日本語で）**：全バックアップ/復元/削除URLの共通入口です。まず一意なタスクIDを発行して台帳に「処理中(pending)」で記録し、ブラウザに即座に番号札を返す準備をします。そして「今どのURL名で呼ばれたか」を逆引きして、実行すべき処理を判断します。

コピー

```
# POSTのみ受け付ける関数ビュー
@api_view(['POST'])
# ファイル・JSON・フォーム形式を解析できる
@parser_classes([MultiPartParser, JSONParser, FormParser])
# 非同期処理の共通入口
def backup(request):
  # タスクID生成
  # 一意なタスクIDを発行（番号札）
  task_id = str(uuid.uuid4())
  # 台帳に「処理中」で記録
  AsyncTask.objects.create(task_id=task_id, status='pending')

  # 開始日（期間指定の処理用）
  start_day = request.data.get('start_day')
  # 終了日
  end_day = request.data.get('end_day')

  # url_name属性取得
  # 今アクセスされたURLパス
  current_path = request.path
  # そのパスを逆引き
  match = resolve(current_path)
  # URLのname（あだ名）を取り出す
  url_name = match.url_name
```

> **▶ ここで初めて出てくるコード**
>
> - **`resolve()`**（Django）：URL の道（ `/api/members/` ）から、どのビューにつながるかを調べます。`resolve(path).url_name` で、URL に付けた名前が分かります。

- **21行目** `@api_view(['POST'])`
- `@api_view`（エーピーアイビュー）は「**関数を** DRFのAPIビューにする」デコレータ。§5のクラスベースと違い、こちらは関数ビューです。`['POST']` で「POSTだけ受け付ける」と指定。
- **22行目** `@parser_classes([MultiPartParser, JSONParser, FormParser])`
- 受信データの形式を3種類解析できるようにします。`MultiPartParser`（マルチパート）は **ファイルアップロード** を扱える解析器。復元(load)ではExcelファイルが送られてくるので必要です。

> ⚠️ **なぜここだけ独自にパーサを指定するの？** `settings.py` の `REST_FRAMEWORK` 設定（§12）では、既定のパーサが `JSONParser` だけになっています。それだとファイルアップロードを受け取れません。そこでこの関数だけ `@parser_classes` でファイル対応の解析器を追加しているのです。

- **25行目** `task_id = str(uuid.uuid4())` … `uuid.uuid4()` で一意なIDを作り、文字列にします。これがタスクの「番号札」。
- **26行目** `AsyncTask.objects.create(task_id=task_id, status='pending')` … 台帳に「このタスクは処理中(pending)」と記録。
- **28〜29行目** … 期間指定の処理用に、開始日・終了日を取り出します。
- **32〜34行目** … `request.path`（今のURLパス）を `resolve` で逆引きし、`url_name`（あだ名）を取り出します。§7.5で見たとおり、URLは全部同じ `backup` 関数を指すので、**この名前で「今どの処理か」を判断** します。

11.3 URL名による処理の振り分け — 36〜189行目

> ▼ **このコードがやること（先に日本語で）**：§11.2で取り出したURL名（kosu_backup, member_load など）を `if/elif` で延々と判定し、それぞれに対応する「実行する関数」と「渡す引数」を決めます。期間指定が必要なものは日付の検証も行い、ファイルアップロードが必要なものは一時ファイルに保存します。

代表的な3パターンだけを抜粋して読みます（残りも同じ形の繰り返しです）。

パターンA：期間指定のバックアップ（kosu_backup）— 36〜41行目

コピー

```
# 工数バックアップなら
if url_name == 'kosu_backup':
  # 日付の妥当性を検査
  error_response = validate_dates(start_day, end_day)
  # 検査でエラーがあれば
  if error_response:
    # その場でエラーを返す
    return error_response
  # 実行する関数を決める
  task_function = generate_kosu_backup
  # 渡す引数（期間）を決める
  args = (start_day, end_day)
```

- **37行目** `validate_dates(start_day, end_day)` … §11.5で見る日付検証関数。問題があればエラー応答が返ります。
- **38〜39行目** … 検証でエラーが出たら、その場で返して中断。
- **40行目** `task_function = generate_kosu_backup` … 実行すべき関数を変数に入れておきます（まだ実行はしない）。

- **41行目** `args = (start_day, end_day)` … その関数に渡す引数を用意。

パターンB：ファイルアップロードを伴う復元（kosu_load）— 48〜61行目

コピー

```
# 工数復元なら
elif url_name == 'kosu_load':
  # アップロードされたファイルを取り出す
  kosu_file = request.FILES.get('file')
  # 一時ファイルパスの入れ物
  temp_file_path = None
  try:
    # 一時xlsxファイルを作る
    with tempfile.NamedTemporaryFile(suffix=".xlsx", delete=False) as temp_file:
      # ファイルを小分け(チャンク)で読み
      for chunk in kosu_file.chunks():
        # 一時ファイルに書き込む
        temp_file.write(chunk)
      # 一時ファイルのパスを覚える
      temp_file_path = temp_file.name
    # 実行する関数を決める
    task_function = load_kosu_file
    # 引数は一時ファイルパス
    args = (temp_file_path,)
  # 書き込み中にエラーが出たら
  except Exception as e:
    # 途中まで作った一時ファイルがあれば
    if temp_file_path and os.path.exists(temp_file_path):
      # 消す（後始末）
      os.remove(temp_file_path)
    # 500を返す
    return JsonResponse({'status': 'error', 'message': f'ファイル書き込みエラー: {str(e)}'}, status=500)
```

▶ ここで初めて出てくるコード

- **`.remove()`**（Python）：`os.remove(ファイルの場所)` は、ファイルを消します。
- **`.FILES`**（Django）：Django の `request.FILES` は、送られたファイルの入れ物です。`request.FILES.get("file")` で 1 つ取り出します。

- **49行目** `request.FILES.get('file')` … `request.FILES` はアップロードされたファイルの保管場所。そこから `'file'` を取り出します。
- **52行目** `with tempfile.NamedTemporaryFile(suffix=".xlsx", delete=False)` … 一時的な `.xlsx` ファイルを作ります。`delete=False` は「withを抜けても自動削除しない」（後で処理に使うため）。
- **53〜54行目** `for chunk in kosu_file.chunks():` … 大きなファイルを `chunks()`（小分け）で少しずつ読み、書き込みます。一度に全部メモリに載せないための工夫です。
- **55行目** `temp_file_path = temp_file.name` … 作った一時ファイルの場所（パス）を覚えておきます。
- **56〜57行目** … 実行する関数とその引数（一時ファイルパス）を決めます。
- **58〜61行目** … もし書き込みでエラーが出たら、途中の一時ファイルを消して(`os.remove`)、500エラーを返します。`f'...{str(e)}'` は **f文字列**（中に変数を埋め込める文字列）。

> **用語: 一時ファイル（temporary file：テンポラリファイル）／チャンク（chunk）** ―一時ファイルは「作業用の使い捨てファイル」。チャンクは「大きなデータを小分けにした塊」。大きなExcelを一気に読むとメモリを食うので、小分け（チャンク）で少しずつ処理します。引っ越しで大きな荷物を一度に運ばず、何回かに分けて運ぶイメージ。

パターンC：引数なしのバックアップ（member_backup など）― 96〜98行目

コピー

```
# 人員バックアップなら
elif url_name == 'member_backup':
  # 実行関数を決める
  task_function = generate_member_backup
  # 引数なし（全件バックアップ）
  args = ()
```

- 人員・チーム・区分などの「全件バックアップ」は期間指定が不要なので、`args = ()`（引数なし）でシンプルです。

最後の else（不明なURL）― 188〜189行目

コピー

```
# どのURL名にも当てはまらなければ
else:
  # 400で返す
  return JsonResponse({'status': 'error', 'message': '無効なタスクタイプです。'}, status=status.HTTP_400_BAD_REQUEST)
```

- どの `if/elif` にも該当しない場合は、不正なリクエストとして400を返します。

11.4 別スレッドでの実行と即時応答 ― 191〜195行目

> **▼ このコードがやること（先に日本語で）**：§11.3で決めた「実行する関数」を、**別スレッド（裏方の作業員）** に渡して走らせます。本体はその完了を待たず、すぐにブラウザへ「番号札（タスクID）」を返します。これでユーザーは待たされません。

コピー

```
# 裏で走らせるスレッドを用意
thread = threading.Thread(target=handle_task, args=(task_id, task_function, *args))
# スレッド開始（裏で処理が走り出す）
thread.start()

# タスクIDを返却し、非同期処理開始を通知
# 番号札（タスクID）を即座に返す
return JsonResponse({'status': 'success', 'task_id': task_id})
```

▶ ここで初めて出てくるコード

- `.Thread()`（Python）：`threading.Thread(target=関数, args=(…))` は、関数を別のスレッド（同時に動く処理）で動かす準備をします。`.start()` で動き出します。重い仕事を裏で動かし、画面をすぐ返すのに使います。
- `.start()`（Python）：`thread.start()` は、準備したスレッドを動かし始めます。呼んだ側は、終わるのを待たずに先へ進みます。

- **191行目** `thread = threading.Thread(target=handle_task, args=(task_id, task_function, *args))`
- `threading.Thread` で「別の作業員（スレッド）」を1人用意します。`target=handle_task` がその作業員にやらせる仕事（§11.7）。`*args` は§11.3で決めた引数を展開して渡します。
- **192行目** `thread.start()` … 作業員に「始めて」と合図。**この瞬間から裏で処理が走り出します。**本体はその完了を待ちません。
- **195行目** `return JsonResponse({'status': 'success', 'task_id': task_id})`
- ブラウザには即座に「受け付けました。番号札はこれです（task_id）」と返します。ブラウザはこの番号札を使って、後から状態を問い合わせます（§11.6）。

▼ **全体の流れ（非同期の旅）**：1. ブラウザ「バックアップして」→ サーバー「番号札123番ね（即返答）」＋裏で処理開始 2. ブラウザ「123番どう？」→ サーバー「まだ処理中(pending)」3. （しばらく後）ブラウザ「123番どう？」→ サーバー「完成(success)！ファイルはここ」4. ブラウザ「ダウンロードする」→ サーバーがファイルを返す

ユーザーは固まらず、待っている間も他の操作ができます。

11.5 validate_dates（日付検証）— 199〜215行目

▼ **このコードがやること（先に日本語で）**：期間指定の処理で、開始日・終了日が正しいかを検査します。空でないか・日付の形式が正しいか・終了日が昨日以前か・開始日が終了日を超えていないかを順にチェックし、問題があればエラー応答を、問題なければ None を返します。

コピー

```
# 日付バリデーション関数
# 日付を検査する関数
def validate_dates(start_day, end_day):
  # 今日の日付を文字列で取得
  today_str = datetime.date.today().strftime('%Y-%m-%d')
  # どちらかが空なら
  if not start_day or not end_day:
    # 400
    return JsonResponse({'status': 'error', 'message': '日付を指定してください。'}, status=status.HTTP_400_BAD_REQUEST)

  try:
    # 終了日を日付型に変換
    end_date_obj = datetime.date.fromisoformat(end_day)
    # 今日を日付型に
    today_date_obj = datetime.date.fromisoformat(today_str)
    # 開始日を日付型に
    start_date_obj = datetime.date.fromisoformat(start_day)
  # 変換に失敗（形式が不正）なら
  except ValueError:
    # 400
    return JsonResponse({'status': 'error', 'message': '日付の形式が不正です。'}, status=status.HTTP_400_BAD_REQUEST)
  # 終了日が今日以降なら
  if end_date_obj >= today_date_obj:
    # 400
    return JsonResponse({'status': 'error', 'message': '昨日の日付までしか指定できません。'}, status=status.HTTP_400_BAD_REQUEST)
  # 開始日が終了日より後なら
  if start_date_obj > end_date_obj:
    # 400
    return JsonResponse({'status': 'error', 'message': '開始日が終了日を超えています。'}, status=status.HTTP_400_BAD_REQUEST)
  # 問題なければNone（合格）
  return None
```

- **201行目** `today_str = datetime.date.today().strftime('%Y-%m-%d')` … 今日の日付を `2026-06-02` 形式の文字列で取得。
- **202〜203行目** … 開始日・終了日のどちらかが空なら400。
- **206〜210行目** … `fromisoformat` で文字列を日付型に変換。変換できなければ（形式不正） `except ValueError` で400。
- **211〜212行目** … 終了日が **今日以降** なら「昨日まで」のルール違反として400。当日のデータはまだ確定していないので、バックアップ対象外という業務ルールです。
- **213〜214行目** … 開始日が終了日より後（逆転）なら400。
- **215行目** `return None` … すべて合格なら `None` を返します。§11.3では「`error_response` が None でなければエラー」と判定していたので、None＝合格の合図です。

11.6 check_task_status（状態確認） — 219〜238行目

> **▼ このコードがやること（先に日本語で）**：ブラウザが番号札（タスクID）を持って「私の処理どうなった？」と聞いてくる窓口です。台帳を見て、成功なら結果ファイルの場所を、エラーなら理由を、まだなら「処理中(202)」を返します。

コピー

```
# GETで状態を問い合わせる
@api_view(['GET'])
# タスク状態確認の窓口
def check_task_status(request):
  # 問い合わせ対象のタスクID
  task_id = request.GET.get('task_id')

  # タスクIDがない場合、エラーを返す
  # 番号札が無ければ
  if not task_id:
    # 400
    return JsonResponse({'status': 'error', 'message': 'タスクIDが指定されていません。'}, status=status.HTTP_400_BAD_REQUEST)

  try:
    # データベースからタスクIDに対応する状態を取得し返す
    # 台帳から該当タスクを取得
    task = AsyncTask.objects.get(task_id=task_id)
    # 成功していたら
    if task.status == 'success':
      # 結果ファイルの場所を返す
      return JsonResponse({'status': 'success', 'file_path': task.result})
    # エラーなら
    elif task.status == 'error':
      # エラー内容を返す
      return JsonResponse({'status': 'error', 'message': task.result})
    # それ以外（処理中）なら
    else:
      # 「処理中(202)」を返す
      return JsonResponse({'status': 'pending'}, status=202)

  # そのIDが台帳に無ければ
  except AsyncTask.DoesNotExist:
    # 400
    return JsonResponse({'status': 'error', 'message': '無効なタスクIDです。'}, status=status.HTTP_400_BAD_REQUEST)
```

▶ ここで初めて出てくるコード

- `.GET`（Django）：`request.GET` は、URL の `?` の後ろの値（`?task_id=abc`）の入れ物です。`request.GET.get('task_id')` で読みます。

- **221行目** `task_id = request.GET.get('task_id')` … 問い合わせ対象のタスクIDを受け取ります。
- **229行目** `task = AsyncTask.objects.get(task_id=task_id)` … 台帳からそのタスクを取得。
- **230〜231行目** … 成功（success）なら、`task.result`（結果ファイルの場所）を返します。
- **232〜233行目** … エラーなら、`task.result`（エラー内容）を返します。
- **234〜235行目** … まだ処理中なら、202（**受理・処理中**） を返します。ブラウザはこれを見て「まだか、また後で聞こう」と判断します。

用語: 202 Accepted（アクセプテッド）「リクエストは受理したが、処理はまだ完了していない」を表す番号。非同期処理で「処理中」を伝えるのに使います。

11.7 handle_task（裏で走る本体） — 268〜302行目

> **▼ このコードがやること（先に日本語で）**：§11.4で別スレッドに渡された「実際の作業」です。指定された関数を実行し、結果を見て、台帳のステータスを成功(success)かエラー(error)に更新します。途中で予期せぬエラーが起きても、台帳にエラーとして記録します。

コピー

```
# 非同期タスク処理（汎用版）
# 裏で走る処理本体
def handle_task(task_id, task_function, *args, **kwargs):
  try:
    # タスク関数を実行し、結果を取得
    # 実際の重い処理を実行
    result = task_function(*args, **kwargs)
    # 結果が「明示的なエラー」かを判定
    is_explicit_error = (
      # 結果がタプルで
      isinstance(result, tuple) and
      # 中身があり
      len(result) > 0 and
      # 先頭が辞書で
      isinstance(result[0], dict) and
      # statusがerrorなら
      result[0].get('status') == 'error'
    )

    # 台帳から該当タスクを取得
    task = AsyncTask.objects.get(task_id=task_id)

    # 明示的エラーなら
    if is_explicit_error:
      # エラーを返した場合
      # エラー辞書を取り出す
      error_dict = result[0]
      # ステータスをerrorに
      task.status = 'error'
      # エラーメッセージを記録
      task.result = error_dict.get('message', 'タスク関数が明示的なエラーを返しました。')
    # 正常終了なら
    else:
      # 正常終了の場合
      # ステータスをsuccessに
      task.status = 'success'
      # 結果（ファイルパス等）を記録
      task.result = result

    # データベースに保存
    # 台帳を更新保存
    task.save()

  # 予期せぬエラーが起きたら
  except Exception as e:
    # 処理中に予期せぬエラーが発生した場合
    try:
      # 台帳を取得
      task = AsyncTask.objects.get(task_id=task_id)
      # errorに
      task.status = 'error'
```

```
            # エラー内容を文字列で記録
            task.result = str(e)
            # 保存
            task.save()
        # 台帳すら見つからなければ
        except AsyncTask.DoesNotExist:
            # ログに出すだけ
            print(f"Error: AsyncTask with id {task_id} not found.")
```

> **▶ ここで初めて出てくるコード**
>
> - `len()`（Python）：長さ（中身の数）を返す関数。`len("abc")` は 3、`len(detail_list)` はリストの中身の数です。

- **271行目** `result = task_function(*args, **kwargs)` … §11.3で決めた実処理（バックアップ生成など）を実行します。ここが時間のかかる本番。**この処理は裏スレッドで走っている** ので、ユーザーは待ちません。
- **272〜277行目** … 結果が「タプルで・先頭が辞書で・statusがerror」という形なら「明示的なエラー」と判定。これはtasks.py 側の関数が「業務的な失敗」を返すときの約束の形です。
- **279行目** `task = AsyncTask.objects.get(task_id=task_id)` … 台帳から該当タスクを取得。
- **281〜285行目** … 明示的エラーなら、ステータスを `'error'`、メッセージを記録。
- **286〜289行目** … 正常終了なら、ステータスを `'success'`、結果（ファイルパスなど）を記録。
- **292行目** `task.save()` … 台帳を更新。これにより、§11.6の状態確認で正しい状態が返るようになります。
- **294〜302行目** … 予期せぬエラー（プログラムの例外）が起きた場合は、台帳に `'error'` とエラー文字列を記録。台帳すら見つからなければ、最後の手段としてログに出すだけにとどめます。

11.8 download_file（ファイルダウンロード） — 242〜263行目

> **▼ このコードがやること（先に日本語で）**：完成したバックアップファイルを、ブラウザにダウンロードさせる窓口です。ファイルを返し終わった後（接続が閉じた後）に、3秒待ってから一時ファイルを自動削除する後始末も仕込んでいます。

コピー

```
# GETでダウンロード
@api_view(['GET'])
# ファイルダウンロードの窓口
def download_file(request):
  # ダウンロード対象のパス
  file_path = request.GET.get('file_path')
  # ファイルをバイナリで開く
  file_handle = open(file_path, 'rb')
  # ダウンロード応答を作る
  response = FileResponse(file_handle, as_attachment=True, filename=os.path.basename(file_path))

  # 遅延削除する内部関数
  def delayed_file_cleanup():
      # 3秒待つ
      time.sleep(3)
      # ファイルがまだあれば
      if os.path.exists(file_path):
        try:
          # 削除
          os.remove(file_path)
        # 削除失敗なら
        except Exception as e:
          # ログに出す
          print(f"Cleanup failed after delay for {file_path}: {e}")

  # 接続が閉じたときに呼ばれる関数
  def cleanup_on_close():
      # 別スレッドで遅延削除
      thread = threading.Thread(target=delayed_file_cleanup)
      # 開始
      thread.start()

  # 応答が閉じたら後始末を呼ぶよう差し替え
  response.close = cleanup_on_close

  # ダウンロード応答を返す
  return response
```

▶ ここで初めて出てくるコード

- **`.basename()`**（Python）：`os.path.basename("C:/a/b/report.xlsx")` は、場所の最後の名前（`report.xlsx`）だけを返します。
- **`.sleep()`**（Python）：`time.sleep(3)` は、3 秒待ちます。

- **244行目** `file_path = request.GET.get('file_path')` … §11.6で受け取ったファイルの場所。
- **245行目** `file_handle = open(file_path, 'rb')` … ファイルを `'rb'`（読み取り・バイナリ）で開きます。Excelなどは文字でなくバイナリなので `b` を付けます。
- **246行目** `response = FileResponse(...)`

- `FileResponse`（ファイルレスポンス）で、ファイルをダウンロードさせる応答を作ります。`as_attachment=True` は「ブラウザで開かず、ダウンロード保存させる」指定。`filename=os.path.basename(file_path)` で保存時のファイル名を指定します。
- **248〜254行目** `delayed_file_cleanup` … 3秒待ってから一時ファイルを削除する内部関数。ダウンロードが終わる前に消すと壊れるので、少し待ちます。
- **257〜259行目** `cleanup_on_close` … 接続が閉じたとき、削除を別スレッドで走らせる関数。
- **261行目** `response.close = cleanup_on_close` … 応答の「閉じる処理」を、後始末つきの関数に差し替えます。これで「ダウンロード完了→自動でファイル削除」が実現します。

> **なぜダウンロード後に消すの？** バックアップファイルは一時的なもので、ためておくとサーバーの容量を圧迫します。ダウンロードが済んだら不要なので、自動で掃除しているのです。「使い終わった皿をすぐ下げる」イメージ。

---

12. 実コード全解説：settings.py（プロジェクト設定）

> **▼ このセクションでわかること:** プロジェクト全体の「設定の集約地」。本章に関わる重要な設定（INSTALLED_APPS, MIDDLEWARE, CORS, REST_FRAMEWORK, SESSION）を中心に読みます。

`settings.py` は「アプリ全体の設定が全部書いてある場所」です。多いので、本章に関係する部分を中心に読みます。

12.1 基本設定 — 8〜23行目

> **▼ このコードがやること（先に日本語で）:** プロジェクトの土台となる場所・秘密鍵・デバッグモード・接続を許可するホスト名を設定します。

コピー

```
# プロジェクトの根っこフォルダの場所
BASE_DIR = Path(__file__).resolve().parent.parent

# 環境変数を読む道具を用意
env = environ.Env()
# .envファイルから秘密の設定を読む
env.read_env(os.path.join(BASE_DIR, '.env'))

# 暗号化などに使う秘密鍵（.envから）
SECRET_KEY=env('SECRET_KEY')

# デバッグモードのオン/オフ（.envから）
DEBUG=env.bool('DEBUG')

# 文字コードはUTF-8
DEFAULT_CHARSET = 'utf-8'

# 接続を許可するホスト名の一覧
ALLOWED_HOSTS = [
    # 本番(Azure)
    'hozen-kosu-react-cwaqashkafgbg5e5.japaneast-01.azurewebsites.net',
    # 開発用
    'localhost',
    # 開発用(自分自身)
    '127.0.0.1'
    ]
```

- **8行目** `BASE_DIR = Path(__file__).resolve().parent.parent` … プロジェクトの根っこの場所を計算。`.env` やデータベースの場所の基準になります。
- **10〜11行目** … `.env` ファイル（秘密の設定を書くファイル）を読み込みます。秘密鍵やDBパスワードはコードに直書きせず、ここから読みます。
- **13行目** `SECRET_KEY=env('SECRET_KEY')` … 暗号化・署名に使う秘密鍵。漏れると危険なので `.env` から読みます。

> **用語: SECRET_KEY（シークレットキー）と .env（ドットエンブ）** SECRET_KEY は、セッションの署名などに使う「アプリの実印」。`.env` は秘密情報（鍵・パスワード）を書く専用ファイルで、Gitに登録しない約束。実印を玄関に貼り出さないのと同じで、コードに直接書きません。

- **15行目** `DEBUG=env.bool('DEBUG')` … デバッグモード。開発中は True（詳しいエラー画面が出る）、本番は False（隠す）。
- **19〜23行目** `ALLOWED_HOSTS` … このアプリに接続してよいホスト名のリスト。許可外からのアクセスは拒否します。

12.2 INSTALLED_APPS ― 25〜38行目

> **▼ このコードがやること（先に日本語で）**：このプロジェクトで使う「機能パック（アプリ）」の一覧です。Django標準の機能に加え、本アプリ独自の `kosu`、API用の `rest_framework`、非同期用の `django_q`、CORS用の `corsheaders` などを有効にします。

コピー

```
# 使う機能パックの一覧
INSTALLED_APPS = [
    # 管理画面
    'django.contrib.admin',
    # 認証
    'django.contrib.auth',
    # コンテンツ型
    'django.contrib.contenttypes',
    # セッション機能
    'django.contrib.sessions',
    # メッセージ
    'django.contrib.messages',
    # 静的ファイル
    'django.contrib.staticfiles',
    # Bootstrap4（管理画面用UI）
    'bootstrap4',
    # 日付ピッカー
    'bootstrap_datepicker_plus',
    # 本アプリ（これが主役）
    'kosu',
    # 非同期タスクキュー
    'django_q',
    # DRF（API機能）
    'rest_framework',
    # CORS対応
    'corsheaders',
]
```

- **34行目** `'kosu',` … 本アプリ。これを登録することで、§10.4の `apps.py`（`KosuConfig`）が読み込まれ、`ready` でシグナルが有効になります。
- **35行目** `'django_q',` … Django-Q。非同期タスクの仕組み（本アプリではバックアップ系で使用）。
- **36行目** `'rest_framework',` … DRF。§5・§6のAPI機能の土台。
- **37行目** `'corsheaders',` … CORS（後述）を扱う部品。

12.3 MIDDLEWARE ― 40〜51行目

> ▼ **このコードがやること（先に日本語で）**：§10で学んだ「関所の列」をここで定義します。リクエストは上から順に、応答は下から逆順に、各関所を通ります。本アプリ独自の CurrentRequestMiddleware は最後に置かれています。

コピー

```
# 関所の列（上から順に通る）
MIDDLEWARE = [
    # ①CORSチェック（最初）
    'corsheaders.middleware.CorsMiddleware',
    # ②セキュリティ
    'django.middleware.security.SecurityMiddleware',
    # ③セッション復元
    'django.contrib.sessions.middleware.SessionMiddleware',
    # ④共通処理
    'django.middleware.common.CommonMiddleware',
    # ⑤CSRFチェック
    'django.middleware.csrf.CsrfViewMiddleware',
    # ⑥認証
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    # ⑦メッセージ
    'django.contrib.messages.middleware.MessageMiddleware',
    # ⑧クリックジャッキング対策
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
    # ⑨静的ファイル配信
    'whitenoise.middleware.WhiteNoiseMiddleware',
    # ⑩独自:リクエスト保管（最後）
    'kosu.middleware.clear_session_middleware.CurrentRequestMiddleware',
]
```

- **41行目** `'corsheaders.middleware.CorsMiddleware'` … CORSチェックを **最初** に置きます（フロントからのアクセス許可を真っ先に判定するため）。
- **43行目** `SessionMiddleware` … セッションを復元する関所。これより後の処理で `request.session` が使えるようになります。§5で `request.session.get('login_No')` を使えたのはこの関所のおかげです。
- **45行目** `CsrfViewMiddleware` … CSRF（後述）対策の関所。
- **50行目** `CurrentRequestMiddleware` … §10.2の独自ミドルウェア。**最後** に置くことで、セッション復元後にリクエストを保管できます。

> ⚠️ **MIDDLEWAREの順番は意味を持つ** 上から順に通るので、順番が大事です。たとえば `SessionMiddleware` より前で `request.session` を使おうとすると、まだ復元されていなくてエラーになります。むやみに並べ替えてはいけません。

12.4 CORS設定 — 53〜59行目

> **▼ このコードがやること（先に日本語で）**：フロント（localhost:3000）とバック（localhost:8000）が別のアドレスでも、ブラウザの安全装置に弾かれず通信できるようにする設定です。

コピー

```
# すべての発信元を許可
CORS_ORIGIN_ALLOW_ALL = True
# クッキー(セッション)の送受信を許可
CORS_ALLOW_CREDENTIALS = True
# 許可する発信元（明示リスト）
CORS_ALLOW_ORIGINS = [
    # Reactの開発サーバー
    'http://localhost:3000',
    # Django
    'http://localhost:8000',
    # Django(自分自身)
    'http://127.0.0.1:8000',
]
```

> **用語: CORS（コルス）とは？** Cross-Origin Resource Sharing（クロスオリジン・リソース・シェアリング）。ブラウザの安全機能で、「別のアドレス（オリジン）への通信」を原則禁止しています。本アプリは開発時、フロントが `localhost:3000`、バックが `localhost:8000` と別アドレスなので、このままでは通信が弾かれます。CORS設定で「この発信元は許可」と明示して、通信できるようにします。「別の建物への立ち入り許可証」のイメージ。

- **53行目** `CORS_ORIGIN_ALLOW_ALL = True` … すべての発信元からのアクセスを許可。開発時の利便性のため。
- **54行目** `CORS_ALLOW_CREDENTIALS = True` … **クッキー（セッションの鍵）を含む通信を許可**。これが無いと、別アドレス間でログイン状態が保てません。本アプリのセッション認証には必須です。
- **55〜59行目** `CORS_ALLOW_ORIGINS` … 許可する発信元の明示リスト。

12.5 REST_FRAMEWORK — 61〜70行目

> **▼ このコードがやること（先に日本語で）**：DRF全体の既定動作を設定します。ページ送りの方式・1ページ件数・受信形式・応答形式を決めます。

コピー

```
# DRFの全体設定
REST_FRAMEWORK = {
    # 既定のページ送り方式
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
    # 既定の1ページ件数
    'PAGE_SIZE': 20,
    # 既定の受信形式解析
    'DEFAULT_PARSER_CLASSES': (
        # JSONのみ
        'rest_framework.parsers.JSONParser',
    ),
    # 既定の応答形式
    'DEFAULT_RENDERER_CLASSES': (
        # JSONで返す
        'rest_framework.renderers.JSONRenderer',
    ),
}
```

- **62行目** `'DEFAULT_PAGINATION_CLASS'` … 既定のページ送り方式。ただし§5.7で見たように、人員ビューは独自の `CustomPagination` を使うので、この既定は上書きされます。
- **63行目** `'PAGE_SIZE': 20` … 既定の1ページ件数。
- **64〜66行目** `'DEFAULT_PARSER_CLASSES'` … 既定では **JSONParserのみ**。だから§11.2でファイルアップロードを扱う `backup` 関数は、わざわざ `@parser_classes` でファイル対応を追加していたのです。
- **67〜69行目** `'DEFAULT_RENDERER_CLASSES'` … 応答は **JSONで返す**。HTMLの管理画面風UIは出さず、純粋なJSONだけを返すAPIにしています。

12.6 セッション設定 — 72〜77行目

> **▼ このコードがやること（先に日本語で）**：ログイン状態を覚えておくセッションの保管先や有効期限、クッキーの安全設定を決めます。

コピー

```
# セッションをデータベースに保存
SESSION_ENGINE = 'django.contrib.sessions.backends.db'
# セッションの有効期限（秒）≒10年
SESSION_COOKIE_AGE = 315360000
# クッキーを送る場面（'None' ＝ ほかのサイトからの通信でも送る）
SESSION_COOKIE_SAMESITE = 'None'
# HTTPS（暗号化した通信）の時だけクッキーを送る
SESSION_COOKIE_SECURE = True
```

- **72行目** `SESSION_ENGINE = 'django.contrib.sessions.backends.db'` … セッションを **データベース** に保存。§5で `request.session.get(...)` が読んでいたのは、このDB保存のセッションです。
- **74行目** `SESSION_COOKIE_AGE = 315360000` … 有効期限。315360000秒 ≒ 10年。長くして「めったにログアウトされない」運用にしています。
- **76行目** `SESSION_COOKIE_SAMESITE = 'None'` … クッキーを送る条件。`'None'` は「ほかのサイトから来た通信にも付けて送る」設定です（`'Lax'` は「ほかのサイトから来た時は、ふつうのリンクの移動の時だけ送る」、`'Strict'` は「同じサイトの中だけ」）。
- **77行目** `SESSION_COOKIE_SECURE = True` … HTTPS（暗号化した通信）の時だけクッキーを送ります。ブラウザは、`'None'` のクッキーを `Secure`（＝`True`）と組み合わせた時しか受け付けないので、76 行目と 77 行目はセットです。本番は HTTPS なので、この設定で動きます。

> ⚠️ **手元で動かす時に、76〜77 行目を書き換えることがある（コミットしない）** 上の 2 行は、GitHub の `master`（＝本番）の設定です。手元の `http://localhost`（暗号化していない通信）で動かすと、ブラウザによっては `Secure` のクッキーが保存されず、ログインしても、すぐログイン画面に戻されることがあります。その時は、手元でだけ 76〜77 行目の先頭に `#` を付けて無効にし、その下に `SESSION_COOKIE_SAMESITE = 'Lax'` と `SESSION_COOKIE_SECURE = False` の 2 行を書いて試します。**書き換えたまま、コミットしないでください。** `master` に push すると、そのまま本番の設定が「暗号化していない通信でもクッキーを送る」に変わります。コミットの前に `git status` を見て、`hozen_another/settings.py` が「modified」と出ていたら `git diff` で確かめ、元に戻します（「000 引き継ぎ書」の §8）。

12.7 その他の重要設定 — 抜粋

コピー

```
# データベース接続設定
DATABASES = {
    'default': {
        # PostgreSQLを使う
        'ENGINE': 'django.db.backends.postgresql',
        # DB名（環境変数から）
        'NAME': os.getenv('DB_NAME', ''),
        # ユーザー
        'USER': os.getenv('DB_USER', ''),
        # パスワード
        'PASSWORD': os.getenv('DB_PASSWORD', ''),
        # ホスト
        'HOST': os.getenv('DB_HOST', ''),
        # ポート
        'PORT': '5432',
    }
}

# CSRFで信頼する発信元
CSRF_TRUSTED_ORIGINS = [
  # 本番
  'https://hozen-kosu-react-cwaqashkafgbg5e5.japaneast-01.azurewebsites.net',
  # 開発
  'http://localhost:8000',
  # 開発
  'http://127.0.0.1:8000',
]

# メディアファイルのURL接頭辞
MEDIA_URL = '/media/'
# メディアファイルの実保存先
MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
```

- **DATABASES** … PostgreSQL（ポストグレス）というデータベースを使い、接続情報は `.env` から読みます。
- **CSRF_TRUSTED_ORIGINS** … CSRF（後述）で信頼する発信元。本番URLと開発URLを登録。
- **MEDIA_URL / MEDIA_ROOT** … §7.6・§11.8で使ったメディアファイル（バックアップExcel）の、URLと実保存先。

---

13. セッション認証とCSRF

最後に、本章で何度も出てきた「セッション認証」と「CSRF」を整理します。

13.1 セッション認証の流れ

本アプリは **セッションベース認証** を使っています。流れは次のとおり。



図7-3: セッションでログインを続ける仕組み。サーバーは中身を保管し、ブラウザは鍵（セッション ID）だけをクッキーで持つ

> **なぜセッションに番号を入れるの？** §5で何度も `request.session.get('login_No')` を呼んで「誰がアクセスしているか」を確認していました。これが成り立つのは、ログイン時にセッションへ番号を保存し、以降のリクエストでクッキー（鍵）から復元しているからです。クッキーは `SessionMiddleware`（§12.3）が処理します。
>
> **用語: クッキー（cookie：クッキー）** ブラウザがサーバーから受け取り、次回以降のリクエストに自動で添付する「小さなメモ」。本アプリではセッションID（鍵）が入っています。手荷物預かり所の「引換券」のイメージ。引換券を見せれば、預けた荷物（セッション）を出してもらえます。

13.2 CSRFとは

> **用語: CSRF（シーサーフ）とは？** Cross-Site Request Forgery（クロスサイト・リクエスト・フォージェリ＝サイト間リクエスト偽造）。悪意のあるサイトが、ログイン中のあなたのクッキーを悪用して、勝手にあなたの名前で操作を送りつける攻撃。たとえば「気づかぬうちに、あなたの権限で人員データが削除される」といった被害。

CSRF対策として、Djangoは「正規の画面からの送信であることを示す合言葉（CSRFトークン）」を要求します。これを `CsrfViewMiddleware`（§12.3）がチェックします。`CSRF_TRUSTED_ORIGINS`（§12.7）に登録した発信元からの送信を信頼します。

なぜCSRF対策が要るの？ セッション認証はクッキーで本人確認するので便利ですが、「クッキーは自動添付される」という性質が悪用されると、本人になりすました操作が通ってしまいます。CSRFトークンという「正規画面でしか手に入らない合言葉」を併用することで、偽の送信を弾きます。「鍵（クッキー）だけでなく、その場で発行される合言葉（トークン）も要求する」二重チェックです。

14. トラブルシューティング

症状	原因	対処
`401 Unauthorized` が返る	未ログイン、またはセッション切れ（`login_No` がセッションに無い）	再ログインする。クッキーが送られているか確認（§13.1）
`403 Forbidden` が返る	ログイン済みだが `authority`（権限）がFalse	管理者に権限付与を依頼。member の authority を確認
`404 Not Found` が返る	指定した `pk`（従業員番号）の人員が存在しない	URLの番号が正しいか、データが消えていないか確認
`400 Bad Request`（番号重複）	登録/更新しようとした従業員番号が既存（§5.9.1, §5.10.3）	別の番号にする
`400`（バリテーションエラー）	送信データがモデルの決まりを満たさない（`is_valid()` がFalse）	必須欄・型・桁数を確認。React側の送信内容を点検
履歴（History）が残らない	`apps.py` の `ready` で `import kosu.signals` が抜けている（§10.4）	`ready` のimportを確認。signals.py が読み込まれているか
履歴の差分が変（前後が混ざる）	スレッド混線、またはキャッシュ漏れ	§9.3.1の安全確認・§9.3.2の保険ルートで概ね防止済み。再現するならロギングで追う
CORSエラー（ブラウザのコンソール）	別オリジン通信が許可されていない（§12.4）	`CORS_ALLOW_ORIGINS` に発信元を追加。`CORS_ALLOW_CREDENTIALS=True` を確認
ログイン状態が保てない	クッキーが送受信されていない	`CORS_ALLOW_CREDENTIALS=True`（§12.4）、React側で `withCredentials` を確認
ファイルアップロードが弾かれる	既定パーサがJSONのみ（§12.5）	そのビューに `@parser_classes([MultiPartParser, ...])` を付ける（§11.2参照）
非同期タスクが「処理中」のまま	裏スレッドでエラー、または `handle_task` が台帳を更新できていない	`check_task_status` の結果と `AsyncTask` テーブルを確認（§11.6, §11.7）
バックアップ日付エラー	終了日が今日以降、または開始>終了（§11.5）	昨日以前の範囲で、開始≦終了になるよう指定
`ImportError`（読み込み失敗）	`.`（同フォルダ）と `..`（上フォルダ）の取り違え	importのドット数を確認（§5.1の注意）

15. 演習問題

以下は、本章の理解を確かめるための問題です。答えは本文中にあります。

問1. `MemberList.get`（§5.3）で、未ログインの場合に返すステータスコードは何番か。また、ログイン済みだが権限が無い場合は何番か。両者の違いを一言で説明せよ。

問2. §5.5で、`annotate` と `Case/When` を使って `shop_order` という一時的な列を作っているのは、何のためか。最終的に一覧でどう並ぶか説明せよ。

問3. §5.9.3（新規）の `serializer = MemberSerializer(data=data)` と、§5.10.3（更新）の `serializer = MemberSerializer(member_instance, data=data)` は、引数の渡し方が違う。それぞれ何モードになり、どう動作が変わるか答えよ。

問4. §6で、全シリアライザが `fields = '__all__'` と書かれている。これはどういう意味で、どんな便利さと注意点があるか。

問5. §7.4の `<int:pk>` について、`/member_update/123/` にアクセスしたとき、`123` はどのように、どの名前でビューに渡されるか。ビュー側の受け取り方も含めて説明せよ。

問6. §9で、ビューのコード（member_views.py）には履歴を残す処理が1行も無いのに、保存・削除のたびに履歴が `History` テーブルに残る。これを実現している仕組み（シグナルと apps.py の ready）を、流れに沿って説明せよ。

問7. §10.2の `CurrentRequestMiddleware` は、なぜ「リクエストをスレッドローカルに保管」する必要があるのか。これが無いと、§9のシグナルで何が困るか説明せよ。

問8. §11で、`backup` 関数（§11.2）は処理の完了を待たずにすぐ `task_id` を返している（§11.4）。なぜ完了を待たないのか。待つ設計だと何が困るか説明せよ。

問9. §12.3で、`SessionMiddleware` は `CurrentRequestMiddleware` より前に並んでいる。この順番が逆だったら何が起きるか、§13.1のセッション復元の流れを踏まえて考えよ。

問10. §13で、セッション認証だけでなく CSRF 対策も必要なのはなぜか。「クッキーが自動添付される」という性質と結びつけて説明せよ。

---

16. この章のまとめ

- **バックエンドは「データを守る裏方の頭脳」**。本アプリは Django + DRF で、画面ではなく **JSON（データ）を返すAPI** として作られている。
- **リクエストの旅** … ブラウザ → MIDDLEWARE（関所） → urls.py（振り分け） → ビュー（処理） → モデル（DB） → シリアライザ（JSON化） → 応答、という一本道。

- **member_views.py** … 4つのクラス（一覧GET・新規POST・更新PUT・削除DELETE）で人員のCRUDを実現。全クラス共通で「①セッションで門番 → ②対象取得 → ③シリアライザで翻訳・検査 → ④適切なステータスコードで応答」というパターンを踏む。
- **serializers.py** … `ModelSerializer` + `fields = '__all__'` で、データ⇔JSONの翻訳を数行で実現。渡し方（`data=` だけ／第1引数つき）で新規・更新を切り替える。
- **urls.py** … URLとビューを結ぶ配線盤。`path('パス', ビュー.as_view(), name='名前')` の3要素。`<int:pk>` でURLから値を受け取る。同じ `backup` 関数を多数のURLが共有し、`name` で処理を分岐。
- **CustomPagination** … 1ページの件数を管理者設定（`menu_row`）から **動的に** 決める。応答に `count`・`page_size`・`results` を含める。
- **signals.py** … `pre_save` / `post_save` / `post_delete` で、保存・削除のたびに **自動で変更履歴を記録**。`get_changes` が「変わった項目だけ」を前後つきで差分計算。ビューには履歴コードが一切無いのに監査が成立する。
- **middleware + apps.py** … `CurrentRequestMiddleware` がリクエストをスレッドに保管し、シグナルから「誰が操作したか」を取得可能にする。`apps.py` の `ready` での `import kosu.signals` がシグナルを有効化する要。
- **asynchronous_views.py** … 重い処理を別スレッドで裏走りさせ、即座にタスクIDを返す **非同期処理**。ブラウザは後から `check_task_status` で状態を問い合わせ、完成したら `download_file` で取得する。
- **settings.py** … プロジェクト全体の設定。MIDDLEWARE の順番・CORS（別オリジン許可）・REST_FRAMEWORK（既定はJSONのみ）・SESSION（DB保存）が本章の要点。
- **セッション認証とCSRF** … クッキー（鍵）で本人確認するのがセッション認証。その「自動添付」を悪用する攻撃を防ぐのが CSRF 対策。両者は二重チェックの関係。
- **ステータスコードの使い分け** … 200成功 / 201作成 / 202処理中 / 204削除成功 / 400入力ミス / 401未認証 / 403権限なし / 404なし。番号で結果が一目で分かる。

これで、本アプリのサーバー側（バックエンド）の全体像と、人員管理機能の隅々まで読めるようになりました。次章では、この章で何度も登場した **モデル（データの設計図）** と、その裏にある **データベース** を詳しく掘り下げます。シリアライザが翻訳し、ビューが操作していた「データそのもの」の正体に迫ります。

➡ 次章: 第8章: データベースとモデル